

Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems

Shaokun Zhang^{*†1} Ming Yin^{*2} Jieyu Zhang³ Jiale Liu¹ Zhiguang Han⁴ Jingyang Zhang² Beibin Li⁵
Chi Wang⁶ Huazheng Wang⁷ Yiran Chen² Qingyun Wu^{1,8}

Abstract

Failure attribution in LLM multi-agent systems—identifying the agent and step responsible for task failures—provides crucial clues for systems debugging but remains underexplored and labor-intensive. In this paper, we propose and formulate a new research area: **automated failure attribution** for LLM multi-agent systems. To support this initiative, we introduce the **Who&When** dataset, comprising extensive failure logs from 127 LLM multi-agent systems with fine-grained annotations linking failures to specific agents and decisive error steps. Using the Who&When, we develop and evaluate three automated failure attribution methods, summarizing their corresponding pros and cons. The best method achieves 53.5% accuracy in identifying failure-responsible agents but only 14.2% in pinpointing failure steps, with some methods performing below random. Even SOTA reasoning models, such as OpenAI o1 and DeepSeek R1, fail to achieve practical usability. These results highlight the task’s complexity and the need for further research in this area. Code and dataset are available in [the public repository](#).

1. Introduction

In recent years, the reframing Large Language Models (LLMs) as agents and built agentic multi-agent systems—composed of interactive, LLM-powered agents collaborating to achieve shared goals—has garnered significant attention (Hong et al., 2023; Li et al., 2023a; Wu et al., 2023). These purposefully designed agentic systems have demon-

^{*}Equal contribution ¹Pennsylvania State University ²Duke University ³University of Washington ⁴Nanyang Technological University ⁵Meta ⁶Google DeepMind ⁷Oregon State University ⁸AG2AI, Inc.. Correspondence to: †Shaokun Zhang <shaokun.zhang@psu.edu>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

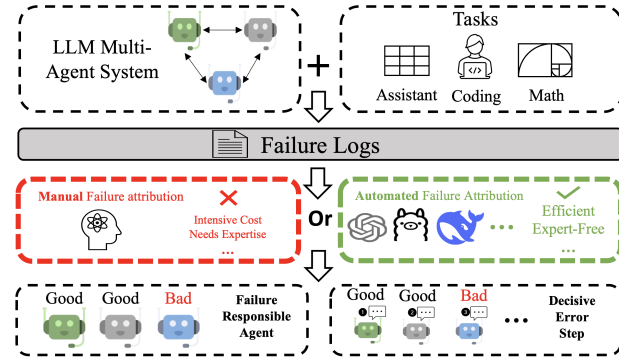


Figure 1. When developing LLMs-powered multi-agent systems, failure attribution—identifying system components responsible for task failures based on evaluation results—has received limited attention in existing research. This process is typically performed manually, demanding substantial labor and specialized expertise. In this study, we explore the potential for automating this process.

strated remarkable potential across various domains, including coding (Hong et al., 2023), scientific discovery (Ghafarollahi & Buehler, 2024), and addressing complex real-world challenges (Fourney et al., 2024).

Once constructed, these systems are typically refined through an iterative process when they fail in specific scenarios: evaluation against established benchmarks, followed by manual failure attribution and system refinement. This cycle repeats until the desired outcomes are achieved. Failure attribution—identifying the components of the system that directly lead to task failures—is a crucial step that serves as the foundation for guiding improvements. Despite its importance, this process is largely overlooked in mainstream research, which typically leaves it as a manual task requiring significant labor, such as analyzing complex historical logs and navigating the technical intricacies of the system. Moreover, mapping benchmark evaluation results to failure components is heavily dependent on domain expertise, imposing additional requirements on practitioners. As systems grow in complexity, this challenge becomes increasingly difficult due to the growing number of components that must be considered when conducting failure attribution.

Previous manual efforts involve a non-straightforward way

哪个智能体导致任务失败，何时发生？——大语言模型多智能体系统中的自动故障归因

张少坤^{*†1} 尹铭^{*2} 张洁宇³ 刘佳乐¹ 韩之光⁴ 张景阳² 李北滨⁵ 王驰⁶ 王华正⁷ 陈一然² 吴青云^{1,8}

摘要

大语言模型多智能体系统中的故障归因——识别对任务失败负责的智能体和步骤——为系统调试提供了重要线索，但仍研究不足且劳动密集。在本文中，我们提出并定义了新的研究领域：大语言模型多智能体系统的自动故障归因。为了支持这一举措，我们引入了Who&When数据集，该数据集包含来自127个大语言模型多智能体系统的大量故障日志，并带有细粒度标注，将失败关联到特定智能体和决定性错误步骤。使用Who&When，我们开发并评估了三种自动故障归因方法，总结了各自的优缺点。最佳方法在识别故障责任智能体方面达到53.5%的准确率，但在定位故障步骤方面仅达到14.2%，部分方法表现甚至低于随机水平。即使是最先进的推理模型，如OpenAI o1和DeepSeek R1，也未能实现实用可用性。这些结果凸显了任务的复杂性以及该领域进一步研究的必要性。代码和数据集在公共仓库中可用。

1. 引言

近年来，将大语言模型（LLMs）重新定义为智能体，并构建了由协作的大语言模型驱动的智能体组成的智能体系统——以实现共同目标——已引起广泛关注（Hong等，2023年；Li等，2023a；Wu等，2023年）。这些有意设计的智能体系统已展示

^{*}同等贡献 ¹宾夕法尼亚州立大学 ²杜克大学 ³华盛顿大学 ⁴南洋理工大学 ⁵Meta ⁶Google DeepMind ⁷俄勒冈州立大学 ⁸AG2AI, Inc.。通信作者：†张少坤<shaokun.zhang@psu.edu>。

国际机器学习会议论文集，温哥华，加拿大。PMLR 267, 2025。版权所有 2025，作者所有。

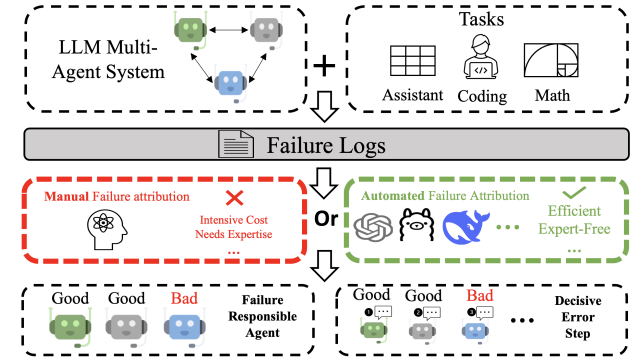


图 1. 在开发大语言模型驱动的多智能体系统时，基于评估结果识别对任务失败负责的系统组件的故障归因，在现有研究中受到的关注有限。该过程通常需要人工执行，需要投入大量人力并具备专门的知识。在本研究中，我们探讨了自动化这一过程的潜力。

出在各个领域的显著潜力，包括编程（Hong等，2023年）、科学发现（Ghafarollahi & Buehler, 2024年）以及解决复杂的现实世界挑战（Fourney等，2024年）。

构建完成后，这些系统通常会在特定场景下失败时，通过一个迭代过程进行优化：首先对照既定基准进行评估，随后进行人工故障归因和系统优化。这一循环会重复进行，直到达到预期结果。故障归因——识别直接导致任务失败的系统组件——是一个关键的步骤，它为指导改进奠定了基础。尽管其重要性，但这一过程在主流研究中被很大程度上忽视了，主流研究通常将其视为一项需要大量人力的人工任务，例如分析复杂的历史日志和应对系统的技术复杂性。此外，将基准评估结果映射到故障组件在很大程度上依赖于领域专业知识，这对从业者提出了额外要求。随着系统复杂度的增加，由于在进行故障归因时必须考虑组件数量的不断增加，这一挑战变得日益困难。

此前的人工工作涉及一种非直截了当的方式

to facilitate failure attribution in multi-agent systems: developing increasingly fine-grained benchmarks, with the hope that more metrics will enable quicker failure attribution (Zhuge et al., 2024). For example, DevAI (Zhuge et al., 2024) introduces a coding benchmark that incorporates diverse delivery requirements, offering a more nuanced evaluation compared to the widely used SWE-Bench (Jimenez et al., 2023), which focuses exclusively on final resolution rates. Despite these advancements, the process of failure attribution based on benchmark results remains a manual process, merely providing more metrics as reference points without fundamentally addressing the underlying challenges. With increasingly comprehensive benchmarks, a fundamental question remains unanswered: **which components of the agentic system require improvement?**

We argue that evaluation and failure attribution should be tightly integrated, adhering to the principle that “evaluation is not an end in itself, but a means to improvement.” (Scriven, 1991) More research efforts should focus on bridging the substantial gap between evaluation results and failure attribution, which currently relies heavily on manual labor. Drawing inspiration from the LLM-as-a-judge paradigm (Gu et al., 2024; Zheng et al., 2023), which leverages LLMs to replace human effort in evaluation, we propose to bridge the gap between evaluation and failure attribution by harnessing the comprehensive judgment capabilities of LLMs. Specifically, we propose and formulate a new research problem: **automated failure attributions** in LLM multi-agent systems. When failures occur under specific scenarios during evaluation, the goal is to automatically identify the components responsible for these failures without human intervention. We believe this research could serve as a substitute for manual failure attribution, enabling human resources to focus on improving system functionality rather than performing time-intensive diagnostics as shown in Figure 1

To advance the research in this area, we introduce the **Who&When** benchmark, comprising extensive failure logs annotated with fine-grained failure details for addressing real-world tasks, where these logs are collected from 127 LLMs-powered multi-agent systems. This benchmark includes both algorithmically generated and hand-crafted multi-agent systems, encompassing a wide range of realistic scenarios. Each failure log is accompanied by meticulous annotations, specifying the failure-responsible agent, the corresponding failure step, and the reasons for failure in plain language. The primary task involves pinpointing the agent most accountable for the failure and the exact step where the error occurred. Predicting the failure-responsible agent serves as a fundamental requirement for failure attribution, given that agents are the basic units of multi-agent systems. Extending this to the specific failure step prediction imposes a higher level of requirement, enabling more fine-grained failure attribution, such as uncovering the spe-

cific reasons behind failures, which can further facilitate targeted system refinements. We believe that Who&When can serve as a foundational resource for driving progress in automated failure attribution research.

Additionally, we construct and evaluate several automated failure attribution approaches on the Who&When. Our findings reveal the strengths and limitations of each method, as well as their performance across different conditions, including model variations, historical context lengths, and the presence or absence of query labels. The results underscore the complexity of using LLMs for failure analysis in multi-agent systems. For example, the best-performing method achieved only 8.77% accuracy in identifying decisive error steps within the hand-crafted agentic system.

2. Problem Formulation: Automated Failure Attribution in Multi-Agent Cooperation

In this section, we introduce decisive errors and formulate the automated failure attribution problem. We adopt the widely-adopted turn-based LLM multi-agent protocol (Hong et al., 2023; Li et al., 2023a; Wu et al., 2023).

Background. Considering a LLMs-powered multi-agent system $\mathcal{M}$ with a group of N agents, denoted as $\mathcal{N} = \{1, 2, \dots, N\}$, that operate at discrete time steps. These agents are taking actions in a turn-based protocol, meaning that exactly one agent performs an action at each time step. Formally, the system is described as:

$$\mathcal{M} = \langle \mathcal{N}, S, A, P, \phi \rangle. \quad (1)$$

Here, S is the set of possible states. A is the global action set; each agent $i \in \mathcal{N}$ can typically perform actions from some subset $A_i \subseteq A$. $\phi(t)$ is a function that indicates which agent is active at time t , thus specifying the turn-based rule. $P(s_{t+1} \mid s_t, a_t, \phi(t))$ is the state-transition probability, given that *only one* agent $\phi(t)$ acts at time t .

We employ $\phi(t)$ to denote the agent that takes an action a_t at time step t . A full trajectory τ can be written as: $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$, where T is a terminal time step or when the system enters a terminating state.

Decisive Error and Objective. We use a tuple (i, t) to denote a mistake in a trajectory, which means agent i is active at time t , and its action a_t is deemed a mistake (e.g., wrong reasoning etc.). A trajectory may contain multiple mistakes, but not all of them result in overall failure. We employ $Z(\tau)$ to denote the result of a trajectory τ .

$$Z(\tau) = \begin{cases} 1, & \text{if the system ultimately fails,} \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

以促进多智能体系统中的故障归因：开发日益细粒度的基准，期望通过更多指标实现更快速的故障归因 (Zhuge等人, 2024)。例如，DevAI (Zhuge等人, 2024) 引入了一个编码基准，其中融合了多样化的交付要求，与主要关注最终解决率的广泛使用的 SWE-Bench (Jimenez等人, 2023) 相比，提供了更为细致的评估。尽管取得了这些进展，但基于基准结果的故障归因过程仍然是一个人工过程，只是提供了更多指标作为参考点，并未从根本上解决潜在的挑战。随着基准日益全面，一个根本问题仍未得到解答：代理系统的哪些组件需要改进？

我们认为，评估与故障归因应紧密整合，并遵循“评估本身并非目的，而是改进手段”的原则 (Scriven, 1991)。目前，评估结果与故障归因之间存在显著差距，且严重依赖人工。未来的研究应致力于弥合这一差距。受“LLM作为裁判”范式 (Gu等人, 2024; Zheng等人, 2023) 的启发，该范式利用大语言模型替代人工进行评估，我们提议通过发挥大语言模型的综合判断能力，来弥合评估与故障归因之间的鸿沟。具体而言，我们提出并定义了一个新的研究问题：大语言模型多智能体系统中的自动故障归因。当在评估过程中发生特定场景下的失败时，其目标是在无需人工干预的情况下，自动识别导致这些失败的组件。我们相信，该研究可作为人工故障归因的替代方案，使人力资源能够专注于提升系统功能，而非执行如图中1所示的高耗时诊断工作。

为了推进该领域的研究，我们引入了Who&When基准，该基准包含大量标注了细粒度故障细节的故障日志，用于解决现实世界任务，这些日志收集自127个大语言模型驱动的多智能体系统。该基准既包含算法生成的多智能体系统，也包含人工打造的多智能体系统，涵盖了广泛的真实场景。每条故障日志都附有细致的标注，以自然语言指明故障责任智能体、相应的故障步骤以及故障原因。主要任务在于精确定位对故障负有最大责任的智能体以及错误发生的确切步骤。由于智能体是多智能体系统的基本单元，预测故障责任智能体是故障归因的基本要求。将此任务延伸至具体故障步骤的预测，则提出了更高的要求，能够实现更细粒度的故障归因，例如揭示特定

失败背后的具体原因，这可以进一步促进有针对性的系统优化。我们认为，Who&When可以作为推动自动故障归因研究进展的基础资源。

此外，我们在Who&When上构建并评估了若干种自动故障归因方法。我们的发现揭示了每种方法的优点和局限性，以及其在不同条件下的表现，包括模型变体、历史上下文长度以及查询标签的存在与否。结果凸显了在大语言模型用于多智能体系统进行故障分析时的复杂性。例如，表现最佳的方法在手工打造的智能体系统中识别决定性错误步骤的准确率仅为8.77%。

2. 问题形式化：多智能体协作中的自动故障归因

在本节中，我们引入决定性错误，并形式化自动故障归因问题。我们采用广泛应用的基于回合的大语言模型多智能体协议 (Hong et al., 2023; Li et al., 2023a; Wu et al., 2023)。

背景。考虑一个由 $\mathcal{M}$ 个智能体组成的、由大语言模型驱动的多智能体系统，记为 $\mathcal{N} = \{1, 2, \dots, N\}$ ，其在离散时间步上运行。这些智能体采用基于轮次的协议执行动作，即在每个时间步中，恰好有一个智能体执行一个动作。形式化地，该系统可描述为：

$$\mathcal{M} = \langle \mathcal{N}, S, A, P, \phi \rangle. \quad (1)$$

此处， S 是可能状态集合。 A 是全局动作集合；每个智能体 $i \in \mathcal{N}$ 通常可以执行来自某个子集 $A_i \subseteq A$ 的动作。 $\phi(t)$ 是一个函数，用于指示在时间 t 时哪个智能体处于激活状态，从而指定轮替规则。 $P(s_{t+1} \mid s_t, a_t, \phi(t))$ 是状态转移概率，其前提是仅有一个智能体 $\phi(t)$ 在时间 t 时执行动作。

我们采用 $\phi(t)$ 来表示在时间步 t 执行动作 a_t 的智能体。完整轨迹 τ 可以表示为： $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$ ，其中 T 是终止时间步，或系统进入终止状态的时刻。

决定性错误与目标。我们使用一个元组 (i, t) 来表示轨迹中的一个错误，这意味着智能体 i 在时间 t 处于活动状态，且其动作 a_t 被视为一个错误（例如，错误推理等）。一条轨迹可能包含多个错误，但并非所有错误都会导致整体故障。我们采用 $Z(\tau)$ 来表示轨迹 τ 的结果。

$$Z(\tau) = \begin{cases} 1, & \text{if the system ultimately fails,} \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Suppose the original trajectory τ is a failure, i.e., $Z(\tau) = 1$. Considering the following scenario, if correcting the mistake made by agent i at time t : we replace a_t with a ”correct” action $\tilde{a}_t$. The steps prior to step t remain unchanged, while the actions following t are adjusted accordingly to ensure correctness. This process generates a modified trajectory:

$$\tau^{(i,t)} = \mathcal{I}_{(i,t)}(\tau), \quad (3)$$

where $\mathcal{I}_{(i,t)}$ denotes the intervention. If in the modified trajectory we obtain $Z(\tau^{(i,t)}) = 0$ (success), then the error (i, t) is said to be a *decisive error*. Formally, we define the *decisive error indicator* $\Delta_{i,t}(\tau)$ as

$$\Delta_{i,t}(\tau) = \begin{cases} 1, & \text{if } Z(\tau) = 1 \text{ and } Z(\tau^{(i,t)}) = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

In words, $\Delta_{i,t}(\tau) = 1 \iff$ Fixing agent i ’s mistake at time t changes $Z(\tau)$ from 1 (fail) to 0 (success). Formally, the decisive error could be defined as agent-time pairs (i^*, t^*) such that $\Delta_{i^*,t^*}(\tau) = 1$, where i^* represents the agent responsible for the system failure, and t^* represents the exact time step at which the critical mistake occurs. We refer to these as the *failure-responsible agent* and the *decisive error step*, respectively across the paper.

In practice, multiple decisive errors may occur within a trajectory. In our study, we address this situation by identifying the earliest error in time as the principal cause of failure. Specifically, we define an objective to determine:

$$\mathcal{C}(\tau) = \{(i, t) \mid \Delta_{i,t}(\tau) = 1\}, \quad (5)$$

$$(i^*, t^*) = \arg \min_{(i,t) \in \mathcal{C}(\tau)} t.$$

which selects the pair (i^*, t^*) yielding the highest decisive error indicator with earliest time step. In this study, the research problem focuses on the automatic identification of the (i^*, t^*) in LLMs-powered multiple agent systems.

3. The Who&When Dataset

To advance research in this area, we introduce a dataset called Who&When. This dataset comprises extensive failure logs from 127 LLM multi-agent systems including both algorithm-generated and human-crafted systems. These logs are carefully annotated with labels that identify the failure-responsible agents and the decisive error steps in agent co-operation directly responsible for problem-solving failures. Additionally, each annotation is supplemented with natural language explanations, culminating in 184 distinct failure annotation tasks. The dataset is specifically designed to detect the failure-reponsible agents (who) and the corresponding steps (when) within each failure log.

Specifically, each instance in Who&When includes the following entry: **(1) Query:** A query from GAIA (Mialon et al., 2023) or AssistantBench (Yoran et al., 2024), describing a real-world question. **(2) Failure log:** The full conversation log of a specific system as it fails to solve the query. **(3) Agentic system information:** For algorithm-generated systems, including system prompts, tools, and agent names, all tailored to this specific query. **(4) Annotations:** An annotation of the agent responsible for task failure, specifying the step where the failure occurred, along with a plain-language explanation of why the failure took place. An example of the instance in this benchmark could be found in Appendix C.

To better reflect our definition of decisive error in Section 2, we design three metrics to evaluate the performance of various failure attribution methods: **(1) Agent-Level Accuracy:** This metric measures the percentage of correctly predicted failure-responsible agents by failure attribution algorithms. **(2) Step-Level Accuracy:** This metric quantifies the percentage of correctly identified decisive error steps. It imposes higher requirements on the algorithms compared to the first metric. **(3) Step-Level Accuracy with Tolerance:** To account for slight deviations, this metric allows a tolerance range for mistake step predictions. If the predicted step falls within the specified tolerance range of the actual mistake step, the prediction is considered correct.

3.1. Agentic Systems Constructions

Who&When includes two types of agentic systems: algorithm-generated agentic systems and one meticulously hand-crafted agentic systems, totaling 127 agentic systems equipped with diverse tools for evaluation.

Algorithm-Generated Agentic Systems. To ensure an adequate number of agentic systems for the Who&When datasets, we first employ the CaptainAgent algorithm (Song et al., 2024) from the AG2 library¹ to automatically generate agentic systems for each data instance sourced from the validation sets of the GAIA (Mialon et al., 2023) and AssistantBench (Yoran et al., 2024) benchmarks. Specifically, it constructs a team of agents tailored to a given task, assigning appropriate agent names, prompts, and necessary tools. The system iteratively optimizes the agents’ configuration until the task is successfully completed. In the Who&When, we select only the final multi-agent configurations, along with the corresponding execution history, as these represent the optimized solutions for each query. All agents within the constructed systems, as well as the CaptainAgent algorithm itself, are based on the GPT-4o on 2024-08-01-preview version. Additionally, since the primary objective of the Who&When is to capture mistakes made by agents that lead to failures in solving real-

¹<https://github.com/ag2ai/ag2>

假设原始轨迹 τ 是失败的，即 $Z(\tau) = 1$ 。考虑以下场景，如果纠正智能体 i 在时间 t 所犯的错误：我们用“正确”的动作 $\tilde{a}_t$ 替换 a_t 。步骤 t 之前的步骤保持不变，而 t 之后的动作则相应调整以确保正确性。此过程生成了一条修改后的轨迹：

$$\tau^{(i,t)} = \mathcal{I}_{(i,t)}(\tau), \quad (3)$$

其中 $\mathcal{I}_{(i,t)}$ 表示干预。如果在修改后的轨迹中我们得到 $Z(\tau^{(i,t)}) = 0$ （成功），则该错误 (i, t) 被称为决定性错误。形式上，我们将决定性错误指示器 $\Delta_{i,t}(\tau)$ 定义为

$$\Delta_{i,t}(\tau) = \begin{cases} 1, & \text{if } Z(\tau) = 1 \text{ and } Z(\tau^{(i,t)}) = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

换句话说， $\Delta_{i,t}(\tau) = 1 \iff$ 在时间 t 纠正智能体 i 的错误，使 $Z(\tau)$ 从 1（失败）变为 0（成功）。形式上，决定性错误可定义为智能体-时间对 (i^*, t^*) ，满足 $\Delta_{i^*,t^*}(\tau) = 1$ ，其中 i^* 表示对系统故障负有责任的智能体， t^* 表示关键错误发生的准确时间步。在全文中，我们将这些分别称为故障责任智能体和决定性错误步骤。

在实际应用中，一条轨迹中可能会出现多个决定性错误。在我们的研究中，我们通过识别时间上最早的错误来应对这一情况，将其确定为失败主因。具体而言，我们定义了一个目标，以确定：

$$\mathcal{C}(\tau) = \{(i, t) \mid \Delta_{i,t}(\tau) = 1\}, \quad (5)$$

$$(i^*, t^*) = \arg \min_{(i,t) \in \mathcal{C}(\tau)} t.$$

该选择器会选择产生最高决定性错误指示器且具有最早时间步的对 (i^*, t^*) 。在本研究中，研究问题聚焦于在大语言模型驱动的多智能体系统中自动识别 (i^*, t^*) 。

3. Who&When 数据集

为了推动该领域的研究，我们引入了一个名为Who&When的数据集。该数据集包含了来自127个大语言模型多智能体系统的广泛失败日志，这些系统既包括算法生成系统，也包括人工构建系统。这些日志经过精心标注，标签用于识别导致问题求解失败的、在智能体协作中直接负有责任的决定性错误步骤，以及失败的负责智能体。此外，每个标注都附有自然语言解释，最终形成了184个不同的失败标注任务。该数据集专门设计用于检测每个失败日志中失败的负责智能体（Who）以及相应的步骤（When）。

具体而言，Who&When中的每个实例包含以下条目：

（1）查询：来自GAIA（Mialon等人，2023）或 AssistantBench（Yoran等人，2024）的查询，描述一个现实世界的问题。（2）失败日志：特定系统在未能解决查询时的完整对话日志。（3）智能体系统信息：对于算法生成的系统，包括系统提示、工具和智能体名称，所有内容均针对该特定查询进行定制。（4）标注：对导致任务失败的责任智能体进行标注，指明失败发生的步骤，并以通俗语言解释失败发生的原因。该基准中实例的示例可参见附录C。

为了更好反映第2节中对决定性错误的定义，我们设计了三项指标来评估各种故障归因方法的性能：（1）智能体级别准确率：该指标衡量故障归因算法正确预测故障责任智能体的百分比。（2）步骤级准确率：该指标量化了正确识别决定性错误步骤的百分比。与第一项指标相比，该指标对算法提出了更高的要求。

（3）带容差的步骤级准确率：为考虑轻微偏差，该指标允许在错误步骤预测中使用容差范围。如果预测步骤落在实际错误步骤的指定容差范围内，则预测被视为正确。

3.1. 智能体系统的构建

Who&When 包含两类智能体系统：算法生成的智能体系统和一个精心打造的智能体系统，总计 127 个配备多样化工具的智能体系统用于评估。

算法生成的智能体系统。为确保 Who&When 数据集拥有充足的智能体系统，我们首先采来自 AG2 库的 CaptainAgent 算法 (Song等人，2024)¹，为源自 GAIA (Mialon等人，2023) 和 AssistantBench (Yoran等人，2024) 基准验证集的每个数据实例自动生成智能体系统。具体而言，该算法针对特定任务构建一个智能体团队，并为每个智能体分配合适的代理名称、提示词以及必要的工具。系统会对智能体配置进行迭代优化，直到任务成功完成为止。在 Who&When 数据集中，我们仅选择最终的多智能体配置及其对应的执行历史，因为它们代表了针对每个查询的最优解决方案。所构建系统中的所有智能体以及 CaptainAgent 算法本身均基于 GPT-4o 的 2024-08-01-preview 版本。此外，鉴于 Who&When 的主要目标是捕捉导致真实智能体系统中任务失败的那些智能体所犯的错误，

¹<https://github.com/ag2ai/ag2>

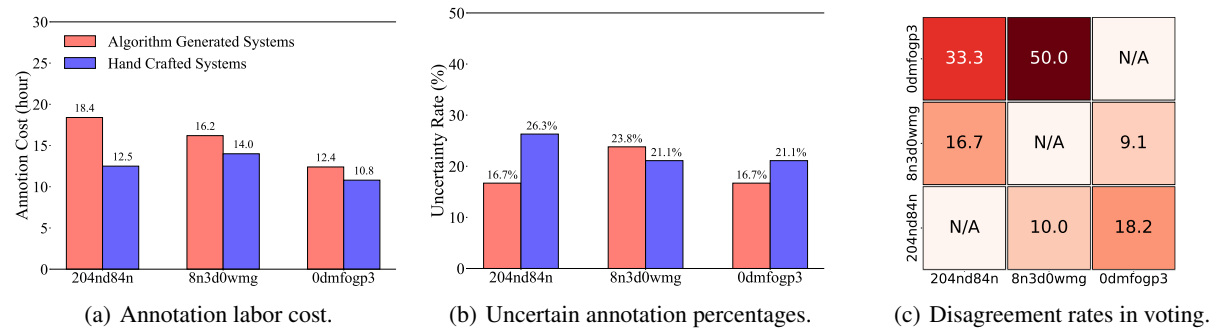


Figure 2. Statistical analysis of the annotation process: (1) Total labor cost for annotations in human hours. (2) The proportion of uncertain annotations to total annotations during the second round. (3) Initial disagreement rates between annotators (note that we make sure to reach a consensus through a careful discussion and voting process afterwards). These results highlight the challenges involved in performing manual failure attribution.

world problems within agentic systems, we retain only those agentic systems that fail to successfully address the queries associated with each data instance from these benchmarks.

Hand-Crafted Agentic Systems. In addition to algorithm-generated systems, Who&When also includes a meticulously hand-crafted, mature multi-agent system, Magnetic-One (Fourney et al., 2024), to ensure the representation of realistic and highly refined agentic systems. Magnetic-One is a generalist agentic system designed to handle a broad range of tasks. It comprises five carefully crafted agents, each specializing in distinct capabilities, such as operating a web browser or navigating local files. We evaluate Magnetic-One using the validation set from the Assistant-Bench (Yoran et al., 2024) benchmark, aggregating its failure logs for subsequent annotation. We also test Magnetic-One on a randomly sampled subset of 30 instances from the GAIA (Mialon et al., 2023), incorporating the corresponding execution failure logs into the dataset. We exclude the rest of the GAIA dataset due to the complexity of annotating the long context logs produced by Magnetic-One.

3.2. Decisive Error Annotation

After obtaining the failure logs of various agentic systems, we introduce an annotation procedure to identify the decisive error failure and decisive error step. To ensure precise annotation, we conduct multiple rounds of annotation performed by three human experts in AI agent (whose identities are anonymized as 0dmfpgp3, 8n3d0wmg, and 204nd84n).

Round I: In the first round, we distribute the failure logs from all agentic systems for each query equally among three experts. To ensure consistency, we provide the experts with a standardized annotation guideline as shown in Appendix F. Each expert is tasked with annotating three elements: the single erroneous agent primarily responsible for the task failure, the specific step at which the error occurred, and the

reasoning behind the mistake in natural language. Additionally, each expert is required to categorize their annotations into two groups: those they are undoubtedly confident are correct and those they have any uncertainty about. **Round II:** In the second round, people are instructed to make an agreement on all the uncertain annotations from Round II. For these uncertain annotations, we engage in a collaborative discussion to reach a consensus. We do not simply follow the principle of majority rule; instead, we aim to ensure that everyone is persuaded and that a consensus is ultimately reached. **Round III:** In the final round, a cross-validation procedure is employed. Each expert is asked to go through another expert’s annotations to assess the consistency of the annotation standards. If any discrepancies or issues with the annotations are identified, the experts engage in further discussion and, if necessary, re-annotate the data according to the established guidelines until a consensus is reached. Incorporating the viewpoints of multiple annotators and ensuring consensus among them, we aim to accurately reach the actual ground truth, as suggested by previous studies (Clemen, 1989; Zhuge et al., 2024).

3.3. Analysis

Annotating the decisive error agent and identifying the specific step of the error is challenging for both non-expert people and domain experts. The annotators must parse complex logs, follow the problem-solving logic of each agent, and assess whether each action is correct or if it misleads the entire problem-solving process. For example, if an agent uses a web browser to gather essential information for problem-solving, annotators must check the browser history and visit each website to determine whether the failure is due to unavailable information on the website or because the agent failed to retrieve it. As shown in Figure 2(a), three annotators spent 30.9, 30.2, and 23.2 human hours, respectively, to complete the annotations. This demonstrates that

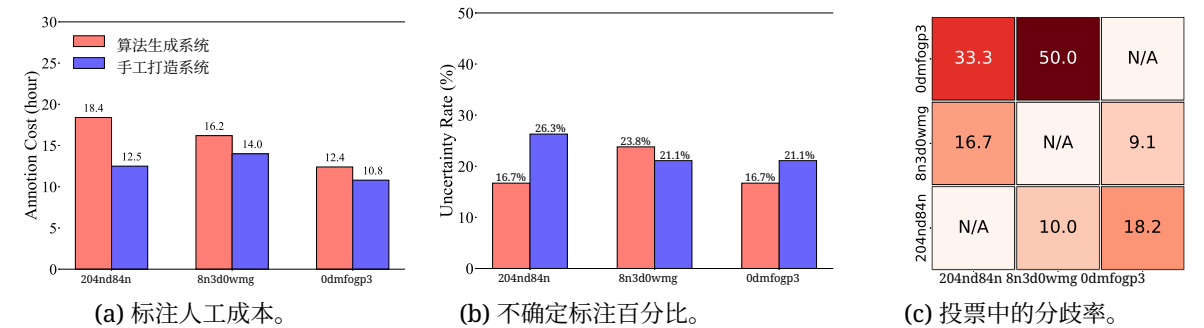


图2. 标注流程的统计分析：（1）人工标注的总人工成本（以小时计）。（2）第二轮中不确定标注占总标注的比例。（3）标注员之间的初始分歧率（请注意，我们确保在此之后通过仔细讨论和投票流程达成共识）。这些结果突显了进行人工故障归因所面临的挑战。

我们仅保留那些未能成功解决这些基准中与每个数据实例相关的查询的智能体系统。

手工打造的智能体系统。除了算法生成的系统外，Who&When还包含一个精心手工打造、成熟的多智能体系统Magnetic-One (Fourney等人, 2024年)，以确保对现实且高度完善的代理系统具有代表性。Magnetic-One是一个通用型代理系统，旨在处理广泛的任務。它由五个精心设计的智能体组成，每个智能体专门负责不同的能力，例如操作网页浏览器或导航本地文件。我们使用Assistant-Bench (Yoran等人, 2024年) 基准测试的验证集对Magnetic-One进行评估，并将其失败日志汇总用于后续标注。我们还在GAIA (Mialon等人, 2023年) 中随机抽取的30个实例子集上测试了Magnetic-One，并将相应的执行故障日志纳入数据集。由于标注Magnetic-One产生的长上下文日志的复杂性，我们排除了GAIA数据集中的其余部分。

3.2. 关键错误标注

在获取各种智能体系统的故障日志后，我们引入了一种标注流程，用于识别决定性错误故障和决定性错误步骤。为确保标注的准确性，我们进行了多轮由三位AI代理人类专家执行的标注（其身份以 0dmfpgp3、8n3d0wmg 和 204nd84n 进行匿名化处理）。第一轮：在第一轮中，我们将每个查询在所有智能体系统中的故障日志平均分配给三位专家。为确保一致性，我们为专家提供了标准化的标注指南，如附录F所示。每位专家的任务是标注三个要素：对任务失败负主要责任的单一错误代理、错误发生的具体步骤，以及

用自然语言描述的该错误背后的推理。此外，每位专家需要将其标注分为两组：他们确信无疑正确的标注和他们对之存有任何不确定性的标注。第二轮：在第二轮中，指示人员就所有来自第二轮的（注：原文如此）不确定标注达成一致意见。对于这些不确定标注，我们进行协作式讨论以达成共识。我们并不简单地遵循多数决规则；相反，我们旨在确保所有人都被说服，并最终达成共识。第三轮：在最后阶段，采用交叉验证程序。要求每位专家审阅另一位专家的标注，以评估标注标准的一致性。如果识别出任何差异或标注问题，专家将进行进一步讨论，并根据既定指南在必要时重新标注数据，直至达成共识。结合多位标注员的观点并确保他们之间达成共识，我们的目标是准确地达到实际真值，正如先前研究 (Clemen, 1989; Zhuge et al., 2024) 所建议的那样。

3.3. 分析

标注决定性错误代理并识别导致错误的具体步骤，对非专家和专业领域的专家而言都具有挑战性。标注员必须解析复杂的日志，遵循每个智能体的问题解决逻辑，并评估每个动作是否正确，或者是否误导了整个问题解决过程。例如，如果一个智能体使用网页浏览器来收集解决问题的关键信息，标注员必须检查浏览器历史记录，并访问每个网站，以确定失败是由于网站上无法获取信息，还是因为智能体未能检索到该信息。如图2(a)所示，三名标注员分别花费了30.9、30.2和23.2个人时来完成标注。这表明

the annotation process is very time-consuming, leading us to consider doing research on automated failure attribution.

Additionally, in many data instances, it’s not just one agent that makes mistakes, but several agents. People need to identify these mistakes and select the most severe ones, which can directly lead to problem-solving failures as formulated in Section 2. Since the severity of mistakes may be subtle and even subjective at times, the process becomes even more difficult. As shown in Figure 2(b), we present the uncertain annotation percentages for three individuals. The uncertain percentages across different annotators range from 15% to 30%. We also visualize the disagreement rates between different individuals when voting on each other’s uncertain data in Figure 2(c). We can see some disagreement remains before discussing to make the agreement, further highlighting the difficulties involved in the annotation process.

4. Can LLMs help identify When and Which agent causes task failures?

As revealed in Section 3.3, detecting the failure-responsible agent and corresponding failure step in agentic system are often subtle, requiring significant human effort. Given these challenges, we were thinking of performing automated failure attribution, using LLMs themselves to detect these errors and provide signal for human to perform essential improvement. In this section, we set up experiments to answer a fundamental question: *Can LLMs help identify when and which agent causes task failures in multi-agent systems?*

4.1. LLMs for Failure Attribution in Agentic Systems

To answer the question mentioned above, we propose three judgement methods for automated failure attribution in agentic systems. Through extensive experiments, we demonstrate that each method has distinct advantages and limitations, and they can be applied either independently or in combination. Furthermore, we analyze the performance of these methods across various scenarios and constraints, highlighting their applicability in different contexts.

(1) All-at-once: An LLM is provided with a query and the complete failure log, and it is tasked with identifying the failure-responsible agent as well as the specific step where the decisive error occurred. **(2) Step-by-step:** An LLM is provided with a query, and the failure log is presented step-by-step. At each step, the LLM is tasked with determining whether a mistake has occurred in the current step. If a mistake is identified, the judging process terminates, and the responsible agent’s name along with the current step number are returned as the output. Otherwise, the process continues until the final step is reached. **(3) Binary search:** Alternatively, this method uses a receptive field approach that lies between the previous two methods. Starting with

the query and the full failure log, the LLM is tasked with determining whether the mistake occurred in the upper half or lower half of the failure logs. Once this decision is made, the LLM is provided with the selected half of the log and the process is repeated iteratively until a single step is identified. The three algorithms and corresponding prompts are detailed in Appendix A and Appendix G.

4.2. Settings

Scenario. We conduct experiments under two distinct settings to simulate various realistic scenarios. **(1) With Ground Truth:** In this setting, the final ground truth of the query that the agentic system is attempting to resolve is available to the LLMs. Our focus here is on the typical AI system development cycle, where it is common practice to use a development dataset with ground truth to identify and debug potential errors in experimental systems. **(2) Without Ground Truth:** In the second setting, the final ground truth of the query is unavailable. In this scenario, LLMs are employed to perform failure attribution in agentic systems based on their running logs. This capability can also be viewed as a form of self-reflection (Huang et al., 2023; Shinn et al., 2024), which contributes to the improvement of multi-agent systems. Throughout this paper, unless otherwise specified, all results are reported as the average accuracy across these two scenarios.

Models. The primary experiments are conducted using the GPT-4o model, unless otherwise specified. Additionally, we also incorporate several other models, including both open-source (such as the Llama and Qwen series) and closed-source models (GPT series), to ensure the consistency of the conclusions drawn from the experiments. Additionally, we employ advanced reasoning models, i.e., OpenAI o1 and DeepSeek R1, to assess the performance of reasoning models on failure attribution tasks. The results of these evaluations are provided in Appendix 4.8.

4.3. Overall Performance

We first perform experiments to compare the performance of three failure attribution methods on Who&When dataset with GPT-4o model. The results are reported on Table 1.

Agent-Level Accuracy Relies on Large Receptive Field. As shown in Table 1, all-at-once significantly outperforms the other two failure attribution methods in agent-level accuracy. Specifically, its agent-level accuracy is 19.13% and 20.69% higher than step-by-step when judging with ground truth, and 25.1% and 20.69% higher when judging without ground truth, respectively. The performance of the binary search method falls between these two approaches.

These results can be attributed to the fact that predicting the

标注流程非常耗时，促使我们考虑开展关于自动故障归因的研究。

此外，在许多数据实例中，出错的不只是一个智能体，而是多个智能体。人员需要识别这些错误，并选择最严重的错误，这些错误可能导致如第2节所定义的问题解决失败。由于错误的严重程度可能很细微甚至有时具有主观性，该过程变得更加困难。如图2(b)所示，我们展示了三个个体的不确定标注百分比。不同标注员之间的不确定百分比在15%到30%之间。我们还在图2(c)中可视化了不同个体在对彼此的不确定数据进行投票时的分歧率。我们可以看到，在讨论达成一致之前仍存在某些分歧，这进一步突显了标注流程中所涉及的困难。

4. 大语言模型能否帮助识别何时以及哪个智能体导致任务失败？

如第3.3节所示，在代理系统中检测故障责任智能体及对应的故障步骤往往较为微妙，需要投入大量人力。鉴于这些挑战，我们考虑通过自动化方式进行任务失败归因，利用大语言模型自身来检测这些错误，并为人类执行必要的改进提供信号。在本节中，我们设计实验以回答一个根本性问题：大语言模型能否帮助识别多智能体系统中何时、由哪个智能体导致了任务失败？

4.1. 智能体系统中用于故障归因的大语言模型

为回答上述问题，我们针对智能体系统中的自动故障归因提出了三种判断方法。通过大量实验，我们证明了每种方法都具有独特的优势和局限性，且可独立或联合应用。此外，我们分析了这些方法在不同场景和约束条件下的表现，强调了其在不同情境中的适用性。

（1）一次性处理：向大语言模型提供查询和完整的失败日志，并要求其识别故障责任智能体以及决定性错误发生的具体步骤。（2）逐步处理：向大语言模型提供查询，并逐步呈现失败日志。在每个步骤中，要求大语言模型判断当前步骤是否发生错误。若识别出错误，则判定流程终止，并返回责任智能体的名称及当前步骤编号作为输出结果。否则，流程继续执行直至到达最后一步。（3）二分搜索：另一种方法是采用介于前两种方法之间的感受野方法。从

输入查询和完整的故障日志后，由大语言模型判断错误发生在故障日志的上半部分还是下半部分。一旦做出该决定，系统将向大语言模型提供所选的一半日志，并重复此过程，直至定位到单个步骤。三种算法及对应的提示词详见附录A和附录G。

4.2. 设置

场景。我们在两种不同的设置下进行实验，以模拟各种真实场景。（1）带真值：在这种设置中，代理系统试图解决的查询的最终真值对大语言模型是可用的。我们在这里关注的是典型的AI系统开发周期，其中使用带有真值的开发数据集来识别和调试实验系统中的潜在错误是一种常见做法。（2）无真值：在第二种设置中，查询的最终真值不可用。在此场景中，基于运行日志，大语言模型被用于在智能体系统中执行故障归因。这种能力也可以被视为一种自我反思（Huang等人，2023；辛等人，2024年），它有助于改进多智能体系统。在本研究中，除非另有说明，所有结果均报告为这两种场景下的平均准确率。

模型。除非另有说明，主要实验均使用 GPT-4o 模型进行。此外，我们还引入了一些其他模型，包括开源模型（如 Llama 和 Qwen 系列）和闭源模型（GPT 系列），以确保从实验中得出的结论的一致性。同时，我们采用先进的推理模型，即 OpenAI o1 和 DeepSeek R1，来评估推理模型在故障归因任务上的性能。这些评估的结果见附录 4.8。

4.3. 总体性能

我们首先进行实验，比较三种故障归因方法在 Who&When 数据集上结合 GPT-4o 模型的性能。结果如表 1 所示。

智能体级别准确率依赖于较大的感受野。如表 1 所示，一次性处理在智能体级别准确率上显著优于其他两种故障归因方法。具体而言，当使用真值进行判断时，其智能体级别准确率分别比逐步处理高出 19.13% 和 20.69%；当无真值进行判断时，则分别高出 25.1% 和 20.69%。二分搜索方法的性能介于这两种方法之间。

这些结果可归因于预测

Title Suppressed Due to Excessive Size				
Agentic Systems Types	With Ground Truth		Without Ground Truth	
	Algorithm Generated	Hand Crafted	Algorithm Generated	Hand Crafted
Random				
Agent-Level Accuracy	29.10	12.00	29.10	12.00
Step-Level Accuracy	19.06	4.16	19.06	4.16
All-at-Once				
Agent-Level Accuracy	54.33	55.17	51.12	53.44
Step-Level Accuracy	12.50	5.26	13.53	3.51
Step-by-Step				
Agent-Level Accuracy	35.20	34.48	26.02	32.75
Step-Level Accuracy	25.51	7.02	15.31	8.77
Binary Search				
Agent-Level Accuracy	44.13	51.72	30.11	36.21
Step-Level Accuracy	23.98	6.90	16.59	6.90

Table 1. Performance of the three failure attribution methods on the Who&When dataset with and without labels, evaluated on the GPT-4o model. For agent-level accuracy, all-at-once outperforms binary search, which in turn surpasses step-by-step. Conversely, for step-level accuracy, step-by-step achieves the best performance, followed by binary search and then all-at-once.

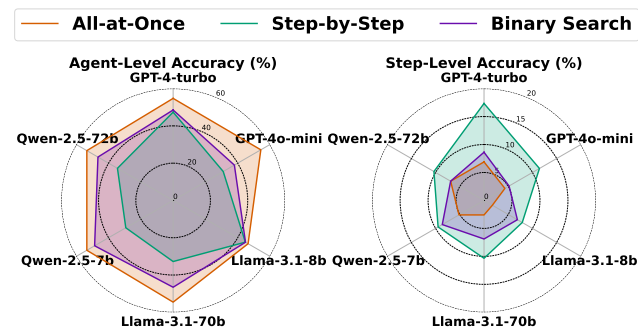


Figure 3. Performance comparison of three failure attribution methods on different models in both two metrics. We found the conclusion is mostly consistent with Table 1.

failure-responsible agent requires the judge LLMs to consider a broader context, including the behaviors of multiple agents. Since all-at-once has access to the entire conversation log when making the final judgment, its prediction of the failure-responsible agent is more accurate. In contrast, the step-by-step method processes the conversation history incrementally, while the final decision can be made with incomplete information, thus resulting in lower performance. Moreover, all failure attribution methods outperform the random baseline, highlighting that these approaches are nontrivial and affirming the necessity of involving LLMs for failure attribution.

Finding 1: Providing broader failure log context enables more accurate agent-level failure attribution by incorporating more complete information.

Fine-Grained Predictions Boost Better Step-Level Accuracy. In terms of step-level accuracy, the all-at-once approach performs obviously the worst, even with its average performance falling below that of random method. This outcome underscores the method’s practical infeasibility. By contrast, the step-by-step approach achieves the highest performance, outperforming other methods in step-level accuracy in 3 out of 4 cases. The performance of the binary search method still falls between these two approaches.

These two results can be attributed to the ‘space-in-the-needle’ problem, where LLMs often struggle to retrieve specific information from long contexts (Nelson et al., 2024). The all-at-once method has access to the largest context when making decisions about the decisive error step, but this can lead to difficulties in pinpointing the exact failure step within a long history. In contrast, the step-by-step method processes the context incrementally, allowing for more focused decision-making. The binary search method performs at a level between these two approaches.

Finding 2: Incrementally processing context enables better step-level failure attribution since LLMs struggle to retrieve information from long contexts.

Impact of Ground Truth on Failure Attribution. We also observed that failure attribution accuracy is higher for all three methods when ground truth are available, compared to when judgments are made without ground truth in all cases in all metrics. Although the answers to users’ queries may not serve as definitive ‘golden labels’ for each agent’s correct behavior, they provide a useful reference signal for the judgment LLMs. For instance, if an agent leads the system in a completely wrong direction, with no possibility of reaching the correct final answer, the label information can directly help alert the judgment LLMs to this error.

标题因尺寸过大而被抑制				
智能体系统类型	带真值		无真值	
	算法生成	手工定制	算法生成	手工定制
随机				
智能体级别准确率	29.10	12.00	29.10	12.00
步骤级准确率	19.06	4.16	19.06	4.16
一次性处理				
智能体级别准确率	54.33	55.17	51.12	53.44
步骤级准确率	12.50	5.26	13.53	3.51
逐步处理				
智能体级别准确率	35.20	34.48	26.02	32.75
步骤级准确率	25.51	7.02	15.31	8.77
二分搜索				
智能体级别准确率	44.13	51.72	30.11	36.21
步骤级准确率	23.98	6.90	16.59	6.90

表1. 三种故障归因方法在带有和不带有标签的Who&When 数据集上的性能，使用GPT-4o模型进行评估。对于智能体级别准确率，一次性处理方法优于二分搜索，而二分搜索又优于逐步处理。相反，对于步骤级准确率，逐步处理取得了最佳性能，其次是二分搜索，最后是一次性处理。

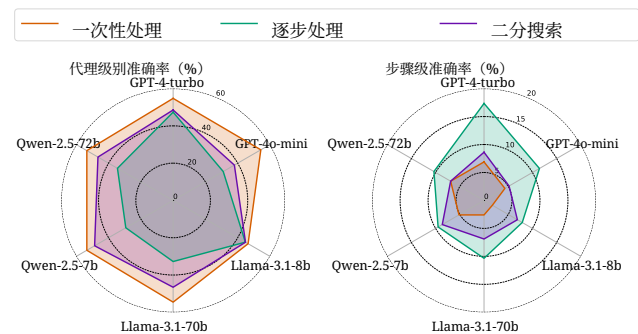


图3. 三种故障归因方法在不同模型上针对两种指标的性能比较。我们发现结论与表1基本一致。

故障责任智能体要求评判大语言模型考虑更广泛的上下文，包括多个智能体的行为。由于一次性处理在做出最终判断时可以访问整个对话日志，因此其对故障责任智能体的预测更为准确。相比之下，逐步方法以增量方式处理对话历史，而最终决策可能在信息不完整的情况下做出，从而导致性能较低。此外，所有故障归因方法均优于随机基线，表明这些方法并非平凡，并证实了在大语言模型故障归因中引入大语言模型的必要性。

发现 1：提供更广泛的失败日志上下文可以通过纳入更完整的信息，实现更准确的智能体级故障归因。

细粒度预测提升了更好的步骤级准确率。在步骤级准确率方面，一次性方法的表现明显最差，即使其平均表现也低于随机方法。这一结果凸显了该方法的实际不可行性。相比之下，逐步处理方法取得了最高的性能，在4种情况中的3种中，其步骤级准确率优于其他方法。二分搜索方法的性能仍然介于这两种方法之间。

这两个结果可以归因于“大海捞针”问题，即大语言模型（LLMs）在处理长上下文时往往难以检索特定信息（Nelson等人，2024年）。一次性方法（all-at-once method）在对决定性错误步骤（decisive error step）做出判断时能够访问最大的上下文，但这可能导致在较长的历史中难以精确定位确切的故障步骤（failure step）。相比之下，逐步方法（step-by-step method）以增量方式处理上下文，从而允许更集中的决策。二分搜索方法（binary search method）的表现介于这两种方法之间。

发现2：增量式处理上下文能够实现更好的步骤级（step-level）故障归因（failure attribution），因为大语言模型（LLMs）在从长上下文（long contexts）中检索信息时存在困难。

真值对故障归因的影响。我们还观察到，在所有指标中，当存在真值时，所有三种方法的失败归因准确率均高于在无真值情况下做出的判断。尽管用户查询的答案可能并不构成各智能体正确行为的“黄金标签”，但它们为判断型大语言模型提供了有用的参考信号。例如，如果一个智能体将系统引向完全错误的方向，且无法获得正确的最终答案，标签信息可以直接帮助判断型大语言模型识别出该错误。

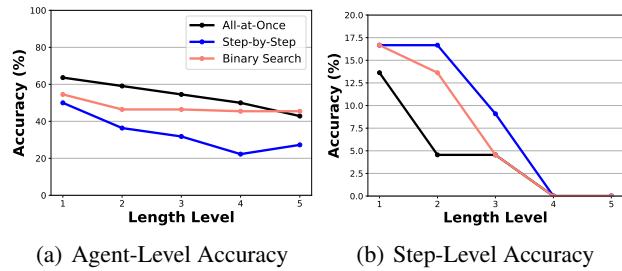


Figure 4. Comparison of three failure attribution methods applied to all failure logs from the hand-crafted systems in the Who&When, evaluated under varying failure log lengths across both metrics.

Without such intervention, the entire system might proceed in the wrong direction without any external warning.

Consistency of Conclusions Across Various LLMs. In addition to the GPT-4o model, we conducted evaluations on other LLMs, including open-source models (e.g., the Llama series and Qwen series) as well as closed-source models (e.g., the GPT series). Due to the significant computational and token costs, we only perform experiments on hand-crafted agentic systems from Who&When which has fewer failure logs. The results of three methods are shown in Figure 3. We found that the phenomena observed in Table 1 hold consistently across different LLMs. Specifically, for agent-level accuracy, the ranking is: all-at-once, followed by binary search, and then step-by-step. Conversely, for step-level accuracy, the ranking is: step-by-step, followed by binary search, and then all-at-once.

Finding 3: The pros and cons of different failure attributions methods in this study are mostly consistent across different LLMs.

4.4. Performance Across Varying Context Lengths

We investigate the relationship between the length of failure logs and the corresponding failure attribution performance. Specifically, the failure logs of hand-crafted agentic systems from the Who&When dataset are divided into five levels, with context length progressively increasing from Level 1 to Level 5. Specifically, Level 1 spans 5–17 steps, Level 2 covers 19–29, Level 3 includes 31–49, Level 4 ranges from 51–91, and Level 5 spans 93–130 steps. Both agent-level and step-level judgment performances across the three evaluation methods are presented in Figure 4. Algorithm-generated systems are excluded from this analysis due to their limited maximum step count of 10, which prevents meaningful divisions of context length.

Our findings indicate that all three methods exhibit a decline in both metrics as context length increases. Notably, step-level accuracy is more sensitive to context length changes

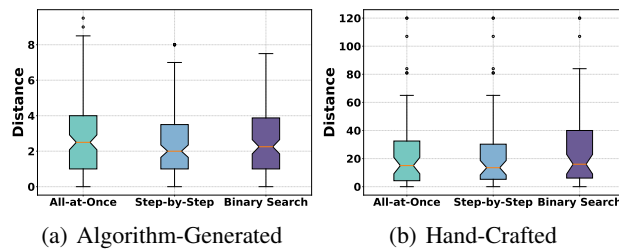


Figure 5. The distances between human-annotated decisive error steps and the predicted steps for each date instance on failure logs from both algorithm-generated and hand-crafted systems.

than agent-level accuracy. Furthermore, the step-by-step performance decline is particularly pronounced compared to the other two. We also analyze the distances between human-annotated decisive error steps and the predicted steps for each data instance, as shown in Figure 5. These results demonstrate that the step-by-step method outperforms the other two methods in accurately predicting the decisive error steps. However, as context length reaches its maximum, all three failure attribution methods converge to near 0%, as shown in Figure 4.

Finding 4: Failure attribution performance declines as context length increases, with step-level accuracy being more sensitive.

4.5. Step-Level Accuracy Under Different Tolerances

Toler.	All-at-Once	Step-by-Step	Binary Search
± 1	12.07	14.66	13.79
± 2	19.83	16.38	18.97
± 3	30.17	18.10	22.41
± 4	37.07	31.90	31.89
± 5	43.10	33.62	36.21

Table 2. Step-level accuracy with different tolerances on the failure logs of hand-crafted agentic systems from Who&When dataset.

In practice, directly identifying the exact decisive error step is not always necessary; it is often sufficient to determine a range of steps where the mistake might occur. In this section, we show the performance of the three failure attribution methods under varying tolerance conditions on the failure logs of hand-crafted agentic systems from the Who&When dataset. Algorithm-generated systems are excluded from this analysis because their maximum step count is limited to 10, and increasing the tolerance would lead to artificially inflated accuracy.

As shown in Table 2, our findings show that step-by-step achieves the highest performance when the tolerance is set to 0 or 1. However, as the tolerance increases, the advantages of all-at-once become more pronounced, while the benefits

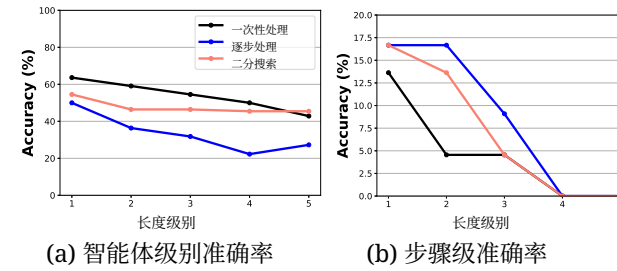


图 4. 针对 Who&When 中手工构建系统的所有故障日志 L 应用的三种故障归因方法的比较，在两种指标下评估不同故障日志长度。

如果没有此类干预，整个系统可能在没有任何外部警告的情况下继续朝着错误的方向发展。

跨不同大语言模型的结论一致性。除了GPT-4o模型外，我们还在其他大语言模型上进行了评估，包括开源模型（例如Llama系列和Qwen系列）以及闭源模型（例如GPT系列）。由于显著的令牌成本，我们仅在Who&When中故障日志较少的手工打造的智能体系统上进行实验。三种方法的结果如图3所示。我们发现，表1中观察到的现象在不同的大语言模型中始终保持一致。具体而言，对于智能体级别准确率，排名依次为：一次性处理，其次是二分搜索，最后是逐步处理。相反，对于步骤级准确率，排名依次为：逐步处理，其次是二分搜索，最后是一次性处理。

发现3：本研究中不同失败归因方法的优缺点在不同大语言模型之间基本保持一致。

4.4. 不同上下文长度下的性能表现

我们研究了故障日志的长度与相应的失败归因性能之间的关系。具体而言，Who&When 数据集中手工打造的智能体系统的故障日志被划分为五个等级，其上下文长度从等级1到等级5逐步增加。具体包括：等级1涵盖5–17步骤，等级2涵盖19–29步骤，等级3涵盖31–49步骤，等级4涵盖51–91步骤，等级5涵盖93–130步骤。图4展示了三种评估方法在智能体级和步骤级上的判定性能。由于算法生成系统的最大步骤数仅为10，无法对上下文长度进行有意义的划分，因此在本分析中被排除。

我们的研究表明，随着上下文长度的增加，所有三种方法在两项指标上均出现下降。值得注意的是，步骤级准确率对上下文长度的变化更为敏感

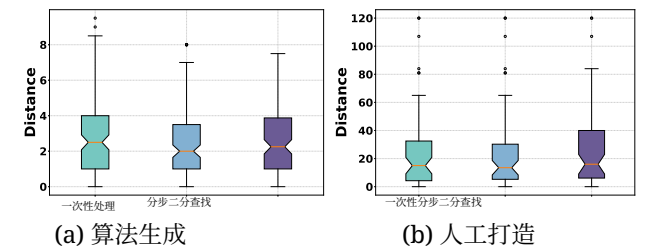


图5. 对于来自算法生成系统和手工打造系统的故障日志，每个日期实例中人工标注的关键错误步骤与预测步骤之间的距离。

高于智能体级别准确率。此外，与其他两种方法相比，逐步方法的性能下降尤为显著。我们还分析了每个数据实例中人工标注的决定性错误步骤与预测步骤之间的距离，如图5所示。这些结果表明，在准确预测决定性错误步骤方面，逐步方法优于其他两种方法。然而，当上下文长度达到最大值时，如图4所示，所有三种故障归因方法的准确率均趋近于0%。

发现4：随着上下文长度的增加，失败归因性能下降，且步骤级准确率的敏感度更高。

4.5. 不同容差下的步骤级准确率

容差	一次性处理	逐步处理	二分搜索
± 1	12.07	14.66	13.79
± 2	19.83	16.38	18.97
± 3	30.17	18.10	22.41
± 4	37.07	31.90	31.89
± 5	43.10	33.62	36.21

表 2. 在 Who&When 数据集的手工打造的智能体系统的故障日志上，不同容差下的步骤级准确率。

在实践中，直接识别出确切的决定性错误步骤并非总是必要的；确定错误可能发生的步骤范围通常就足够了。在本节中，我们展示了三种故障归因方法在Who&When 数据集的手工打造的智能体系统的故障日志上，在不同容差条件下的表现。算法生成的系统被排除在此次分析之外，因为其最大步骤数限制为10，且增加容差会导致准确率人为地虚高。

如表2所示，我们的发现表明，当容差设置为0或1时，逐步处理实现了最高的性能。然而，随着容差的增加，一次性处理的优势变得更加明显，而逐步处理的好处

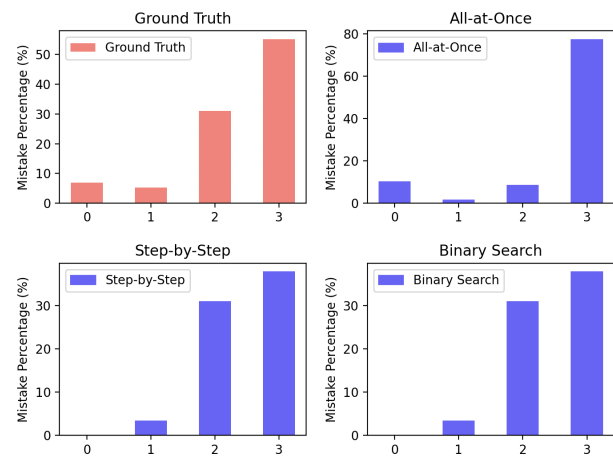


Figure 6. Histogram of the actual and predicted failure-responsible agents for all three methods. We present only the failure logs of hand-crafted systems in Who&When to aggregate the largest number of results for one multi-agent system. Number 0, 1, 2, 3 represents Assistant, FileSurfer, Orchestrator and WebSurfer respectively.

of step-by-step diminish. Compared to all-at-once, step-by-step demonstrates better alignment with accurate predictions when high precision is required.

Finding 5: Allowing tolerance in failure attribution enables broader context processing methods to achieve competitive step-level accuracy.

4.6. A Statistical Viewpoint on Failure Attribution

This study primarily perform experiments on single-data-level failure attribution in LLM-powered multi-agent systems, i.e., identifying the specific component (referred to as the failure-responsible agent) and the precise location (the decisive error step) responsible for task failure in a single data instance. This practice indeed mirrors human procedures for failure attribution and could serve as a foundational tool for deriving statistical-level conclusions. Therefore, we think of whether these methods could be applied to entire datasets to extract meaningful statistical results.

In Figure 6, we show the histogram of actual and the predicted failure-responsible agents for all three methods. We only show the failure logs of hand-crafted systems from Who&When to aggregate the largest number of results for one system type. We observe that the single agent that make the most decisive errors predicted by all methods to be consistent with the ground truth (agent 3). Moreover, the top two failure-responsible agents predicted by three methods are also consistent with the ground truth in most cases (2 out of 3). These experiments demonstrate that, although the instance-level failure attribution results are not highly

positive, all three methods still yield meaningful insights from a statistical perspective. In practice, these statistical results provide a more actionable basis for system refinement compared to focusing solely on single data instances.

Finding 6: The three baseline methods are more effective at performing failure attribution at a statistical level than at an instance level.

4.7. Can We Combine Multiple Failure Attribution Methods?

Metrics	Cost Token Num	Agent-Level Accuracy	Step-Level Accuracy
Binary Search	34,659	43.97	6.90
△ All-at-Once	17,106	57.02	4.39
□ Step-by-Step	87,720	35.96	7.90
Hybrid Method (□&△)	149,177	57.02	12.28

Table 3. Comparison of the three failure attribution methods with a hybrid approach that combines all-at-once and step-by-step on the failure logs of hand-crafted systems from the Who&When dataset. The hybrid method achieves the highest performance in both two metrics but incurs the highest token costs.

We then investigate whether a hybrid method could leverage the advantages of both two different methods, all-at-once and step-by-step. The former excels at failure-responsible agent predictions, while the latter is better at accurately predicting the decisive error step. Specifically, we start by prompting all-at-once to predict the failure-responsible agent and then use step-by-step to detect the mistake step in the actions step taken by the identified failure-responsible agent. To evaluate this, we perform experiments on the hand-crafted systems from the Who&When dataset considering the token cost. The results are shown on Table 3.

We observe that the hybrid method outperforms all methods in step-level accuracy. This improvement is attributed to the all-at-once narrowing the range of possible failure steps by excluding action steps taken by other agents, thereby significantly reducing the difficulty of prediction for step-by-step. However, the hybrid method comes with a notable drawback: it requires running two algorithms sequentially. Compared to making judgments with a single algorithm, this approach incurs higher computational costs.

Finding 7: Combining different failure attribution methods allows leveraging their respective strengths for better performance.

4.8. Strong Reasoning Model for Automated Failure Attributions

We finally examine whether reasoning models OpenAI o1 and DeepSeek R1 (DeepSeek-AI, 2025) can enhance the automated failure attribution process. However, the original

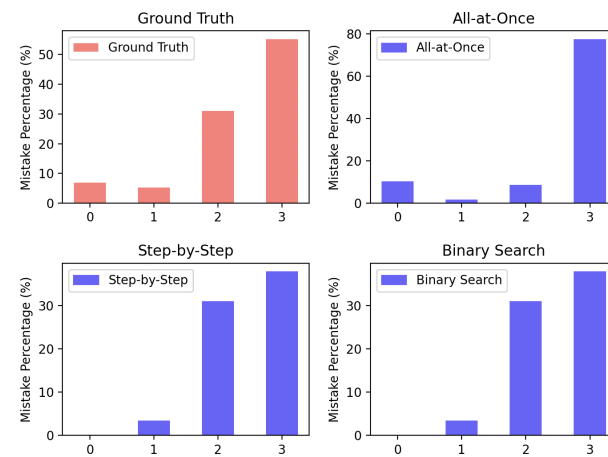


图6. 三种方法中实际与预测的故障责任智能体的直方图。在 Who&When 中，我们仅展示手工构建系统的故障日志，以便为单一多智能体系统汇总最大数量的结果。编号0、1、2、3分别代表助理、文件浏览器、编排器和网页浏览器。

则会减弱。与一次性处理相比，逐步处理在需要高精度时，与准确预测的对齐性更好。

发现5：在失败归因中引入容差，使更广泛上下文处理方法能够达到具有竞争力的步骤级准确率。

4.6. 故障归因的统计视角

本研究主要在大语言模型驱动的多智能体系统上开展单数据实例层面的故障归因实验，即识别导致单数据实例任务失败的特定组件（称为故障责任智能体）以及确切位置（决定性错误步骤）。这种做法确实与人类的故障归因流程相一致，并可作为得出统计层面结论的基础工具。因此，我们考虑这些方法是否可应用于整个数据集，以提取有意义的统计结果。

在图 6 中，我们展示了针对所有三种方法的实际故障责任智能体与预测故障责任智能体的直方图。我们仅展示来自 Who&When 的手工构建系统的故障日志，以便为单一系统类型汇总最大数量的结果。我们观察到，所有方法预测的犯下最多决定性错误的单个智能体与真值（智能体 3）保持一致。此外，三种方法预测的前两个故障责任智能体在大多数情况下也与真值一致（3 个中有 2 个）。这些实验表明，尽管实例级故障归因的结果并不高度

积极，但三种方法仍然能从统计视角提供有意义的洞察。在实践中，与仅关注单数据实例相比，这些统计结果为系统优化提供了更具可操作性的基础。

发现 6：三种基准方法在统计层面进行故障归因比在实例层面更有效。

4.7. 我们能否结合多种故障归因方法？

指标	Cost 令牌数量	智能体级别 准确率	步骤级 准确率
二分搜索	34,659	43.97	6.90
△ 一次性处理	17,106	57.02	4.39
□ 逐步处理	87,720	35.96	7.90
混合方法 (□&△)	149,177	57.02	12.28

表 3. 结合一次性处理和逐步处理的混合方法在 Who&When 数据集中手工构建系统的故障日志上，对三种故障归因方法进行的比较。该混合方法在两项指标上均取得了最高的性能，但同时也产生了最高的令牌成本。

随后，我们探讨混合方法是否能够同时利用一次性处理和逐步处理这两种不同方法的优势。前者擅长预测故障责任智能体，而后者在准确预测决定性错误步骤方面更具优势。具体而言，我们首先使用提示词让一次性处理预测故障责任智能体，然后使用逐步处理来检测该已识别的故障责任智能体所执行的动作步骤中的错误步骤。为了评估这一点，我们考虑了令牌成本，在 Who&When 数据集中的手工构建系统上进行了实验。结果如表 3 所示。

我们观察到，混合方法在步骤级准确率上优于所有其他方法。这一提升归功于一次性处理通过排除其他智能体执行的动作步骤，缩小了可能故障步骤的范围，从而显著降低了逐步处理的预测难度。然而，混合方法也存在一个明显的缺点：它需要顺序运行两个算法。与使用单一算法做出判断相比，这种方法会产生更高的计算成本。

发现 7：结合不同的故障归因方法能够利用各自的优势以实现更好的性能。

4.8. 用于自动故障归因的强推理模型

我们最终考察推理模型 OpenAI o1 和 DeepSeek R1 (DeepSeek-AI, 2025) 是否能增强自动故障归因过程。然而，原始的

Title Suppressed Due to Excessive Size						
	GPT-4o		OpenAI o1		DeepSeek R1	
Accuracy	Agent-Level	Step-Level	Agent-Level	Step-Level	Agent-Level	Step-Level
All-at-Once	54.31	4.39	41.38	10.34	56.90	3.45
Step-by-Step	33.62	7.90	36.21	13.79	32.76	6.90

Table 4. The performance of the automated failure attribution methods with reasoning mechanism with strong reasoning models.

prompt used in our experiments was flagged by OpenAI’s policy as violating usage guidelines. Therefore, we implemented minor modifications to the prompt while preserving its original intent. For DeepSeek R1, we employed the same prompt as used in other experiments to ensure consistency. The results are shown in Table 4.8. We don’t include binary search because it doesn’t include reasoning mechanisms in their prompt. We perform experiments on hand-crafted agentic systems of Who&When. The results indicate that stronger reasoning models do not necessarily outperform standard models. Although it provides some improvement, but still far from practical usability. For instance, DeepSeek R1 underperforms GPT-4o in three out of four cases, and OpenAI o1 fails to consistently surpass GPT-4o across all metrics. These findings highlight the inherent challenges of failure attribution. In contrast, integrating reasoning mechanisms into the prompt yields significant performance improvements across all metrics and cases, as shown in Figure 7. This demonstrates that replacing the base model alone does not guarantee better outcomes.

5. Related Works

LLM Multi-Agent Systems. An emerging research focus examines using LLMs (Achiam et al., 2023; Wang et al., 2024) as central controllers to develop LLM agents that interact with the external world beyond text domains (Deng et al., 2024; Xie et al., 2024; Zhang et al., 2024b; 2025). While single-agent systems (Yao et al., 2022; Zhang et al., 2023a; 2024a) excel in specific tasks, they struggle with challenges requiring collaboration and collective intelligence. To address this, studies have explored LLM-powered multi-agent systems, where multiple interactive agents work concurrently (Hong et al., 2023; Li et al., 2023a). These systems leverage the specialized skills and roles of individual agents, enabling collaborative problem-solving for complex tasks by simulating real-world cooperation patterns.

LLM for Judging. Numerous studies have explored the use of large language models (LLMs) as evaluators to assess various tasks based on pre-defined standards (Fu et al., 2023; Gu et al., 2024; Hu et al., 2024; Li et al., 2023b; Liu et al., 2023; Thakur et al., 2024). For instance, Chan et al. (2023); Zheng et al. (2023) utilize LLMs to evaluate the performance of LLMs in chat conversation scenarios, which would otherwise incur significant labor costs if performed by humans. Another notable example is Miao et al. (2023);

van Schaik & Pugh (2024), who employ LLMs as evaluators in the context of text summarization which also heavily relies on human efforts. In the field of agentic systems, related research includes Shinn et al. (2024), who adopt the concept of LLMs-as-judges to analyze task feedback signals and guide corrective actions. Similarly, Zhuge et al. (2024) demonstrate the use of LLMs to provide detailed evaluations of agentic systems within their proposed DevAI dataset. Despite these advancements, failure attribution remains a manual process, with evaluation results serving only as a reference for such attributions

Reward Models Most reward models (RMs) are designed either to predict human preference rankings for outputs generated by large language models (Zhong et al., 2025) or to evaluate the reasoning process step by step, rather than assessing only the final answer (Cui et al., 2025; Lightman et al., 2023; Wang et al., 2023; Zheng et al., 2024). A number of studies have proposed training process-level reward models that evaluate the correctness of intermediate reasoning steps produced by a single LLM (Cui et al., 2025; Lightman et al., 2023). For instance, Math-Shepherd (Wang et al., 2023) employs automatically generated supervision data to assign reward scores to each step in solving mathematical problems. Similarly, ProcessBench introduces a benchmark of step-by-step solutions annotated by human experts, identifying the location of errors within mathematical problem-solving processes. In this setting, models are tasked with detecting the earliest erroneous step or confirming that the entire solution is correct. However, these works focus primarily on constructing reward models for evaluating the outputs of individual LLMs, rather than identifying the errors in complex agentic systems.

6. Conclusion

In this study, we propose and formulate a new research area: automated failure attribution in LLM multi-agent systems, an area that has been largely overlooked in current research. To advance this field, we introduce the Who&When dataset, which consists of 127 multi-agent systems with extensive failure logs meticulously annotated with failure details. Furthermore, we develop and evaluate three automated failure attribution methods, highlighting the challenges and complexities of this task. Our findings underscore the significant difficulty of automated failure attribution and emphasize the urgent need for further research in this emerging area.

标题因尺寸过大而被抑制						
	GPT-4o		OpenAI o1		DeepSeek R1	
准确率	智能体级	步骤级	智能体级	步骤级	智能体级	步骤级
一次性处理	54.31	4.39	41.38	10.34	56.90	3.45
逐步处理	33.62	7.90	36.21	13.79	32.76	6.90

表4。采用强推理模型、具备推理机制的自动化失败归因方法的性能。

提示词在我们的实验中被 OpenAI 的政策判定为违反使用指南。因此，我们对提示词进行了轻微修改，同时保留其原始意图。对于 DeepSeek R1，我们采用了与其他实验相同的提示词，以确保一致性。结果如表 4.8 所示。我们未包含二分搜索，因为其提示词中未包含推理机制。我们在 Who&When 的手工打造的智能体系统上执行实验。结果表明，更强的推理模型并不一定优于标准模型。尽管它带来了一些改进，但仍未达到实用可用性。例如，DeepSeek R1 在四个案例中有三个表现不如 GPT-4o，且 OpenAI o1 在所有指标上都无法稳定地超越 GPT-4o。这些发现凸显了故障归因的固有挑战。相比之下，将推理机制集成到提示词中，在所有指标和案例中均带来了显著的性能提升，如图 7 所示。这表明，仅替换基础模型并不能保证更好的结果。

5. 相关工作

大语言模型多智能体系统。一个新兴的研究重点在于探讨如何利用大语言模型 (Achiam等人，2023年；王等人，2024年) 作为核心控制器，开发能够与文本领域之外的外部世界进行交互的大语言模型智能体 (Deng等人，2024年；Xie等人，2024年；Zhang等人，2024b年；2025年)。尽管单智能体系统 (Yao等人，2022年；Zhang等人，2023a年；2024a年) 在特定任务上表现出色，但它们在应对需要协作和集体智慧的挑战时往往力不从心。为解决这一问题，相关研究探索了由大语言模型驱动的多智能体系统，其中多个智能体并发地进行交互 (Hong等人，2023年；Li等人，2023a年)。这些系统充分利用各个智能体的专业技能与角色分工，通过模拟现实世界的合作模式，实现对复杂任务的协作解决问题。

LLM用于评判。众多研究已经探索了将大语言模型 (LLMs) 作为评估器，根据预定义标准评估各种任务的方法 (Fu等人，2023；Gu等人，2024年；Hu等人，2024；Li等人，2023b；Liu等人，2023；Thakur等人，2024)。例如，Chan等人 (2023) ；郑等人 (2023) 利用大语言模型来评估大语言模型在聊天对话场景中的表现，而这一工作如果由人类执行将产生显著的人工成本。另一个值得注意的例子是Miao等人 (2023) ；

万·施凯克与帕格 (2024)，他们在文本摘要的背景下将大语言模型作为评估器，而这一工作同样高度依赖人工努力。在智能体系统领域，相关研究包括辛等人 (2024)，他们采用LLM作为评判者的概念来分析任务反馈信号并指导纠正措施。类似地，诸葛等人 (2024) 展示了在大语言模型在其提出的DevAI数据集中对智能体系统进行详细评估的应用。尽管取得了这些进展，但故障归因仍然是一个手动过程，评估结果仅作为此类归因的参考。

奖励模型 大多数奖励模型 (RM) 的设计目的是预测人类对大语言模型生成输出的偏好排名 (钟等人，2025年)，或者逐步评估推理过程，而不仅仅是评估最终答案 (崔等人，2025年；莱特曼等人，2023年；王等人，2023年；郑等人，2024年)。许多研究提出训练过程级奖励模型，以评估单个大语言模型产生的中间推理步骤的正确性 (崔等人，2025年；莱特曼等人，2023年)。例如，Math-Shepherd (王等人，2023年) 利用自动生成的监督数据为求解数学问题过程中的每个步骤分配奖励分数。同样，ProcessBench 引入了一个由人类专家标注的逐步解题基准，用于识别数学解题过程中错误所在的位置。在该设定中，模型的任务是检测最早出现的错误步骤，或确认整个解答是正确的。然而，这些工作主要专注于构建用于评估单个大语言模型输出的奖励模型，而不是识别复杂智能体系统中的错误。

6. 结论

本研究提出并构建了一个新的研究领域：大语言模型多智能体系统中的自动故障归因，该领域在现有研究中尚未得到充分关注。为推动该领域的发展，我们引入了Who&When数据集，该数据集包含127个多智能体系统，并对大量故障日志进行了细致标注，记录了故障详情。此外，我们开发并评估了三种自动化失败归因方法，凸显了该任务面临的挑战与复杂性。我们的研究结果强调了自动故障归因的重大难度，并指出了这一新兴领域亟需进一步研究的迫切性。

Title Suppressed Due to Excessive Size	
Impact Statement	
Our approach has societal implications, both positive and negative. On the positive side, our work contributes to the efficient development of multi-agent systems powered by LLMs, enabling their application across a wide range of domains. Incorporating mechanisms for failure attribution and conduct corresponding improvement, these advancements have the potential to enhance LLM multi-agent systems significantly. However, the work also introduces potential risks. For instance, granting these systems the ability to modify external environments, such as executing code on computers, could lead to unintended consequences.	
References	
Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. <i>arXiv preprint arXiv:2303.08774</i> , 2023.	
Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., Zhang, S., Fu, J., and Liu, Z. Chateval: Towards better llm-based evaluators through multi-agent debate. <i>International Conference on Learning Representations</i> , 2023.	
Clemen, R. T. Combining forecasts: A review and annotated bibliography. <i>International journal of forecasting</i> , 1989.	
Cui, G., Yuan, L., Wang, Z., Wang, H., Li, W., He, B., Fan, Y., Yu, T., Xu, Q., Chen, W., et al. Process reinforcement through implicit rewards. <i>arXiv preprint arXiv:2502.01456</i> , 2025.	
DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning deepseek-ai. <i>arXiv preprint arXiv:2501.12948</i> , 2025.	
Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2web: Towards a generalist agent for the web. <i>Advances in Neural Information Processing Systems</i> , 2024.	
Fourney, A., Bansal, G., Mozannar, H., Tan, C., Salinas, E., Niedtner, F., Proebsting, G., Bassman, G., Gerrits, J., Alber, J., et al. Magentic-one: A generalist multi-agent system for solving complex tasks. <i>arXiv preprint arXiv:2411.04468</i> , 2024.	
Fu, J., Ng, S.-K., Jiang, Z., and Liu, P. Gptscore: Evaluate as you desire. <i>Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies</i> , 2023.	
Ghafarollahi, A. and Buehler, M. J. Sciagents: Automating scientific discovery through multi-agent intelligent graph reasoning. <i>arXiv preprint arXiv:2409.05556</i> , 2024.	
Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., et al. A survey on llm-as-a-judge. <i>arXiv preprint arXiv:2411.15594</i> , 2024.	
Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., et al. Metagpt: Meta programming for multi-agent collaborative framework. <i>International Conference on Learning Representations</i> , 2023.	
Hu, Z., Zhang, J., Xiong, Z., Ratner, A., Xiong, H., and Krishna, R. Language model preference evaluation with multiple weak evaluators. <i>arXiv preprint arXiv:2410.12869</i> , 2024.	
Huang, J. and Chang, K. C.-C. Towards reasoning in large language models: A survey. In <i>Findings of the Association for Computational Linguistics: ACL 2023</i> , 2022.	
Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. Large language models cannot self-correct reasoning yet. <i>arXiv preprint arXiv:2310.01798</i> , 2023.	
Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. Swe-bench: Can language models resolve real-world github issues? <i>International Conference on Learning Representations</i> , 2023.	
Li, G., Hammoud, H., Itani, H., Khizbullin, D., and Ghanem, B. Camel: Communicative agents for” mind” exploration of large language model society. <i>Advances in Neural Information Processing Systems</i> , 36:51991–52008, 2023a.	
Li, X., Zhang, T., Dubois, Y., Taori, R., Gulrajani, I., Guestrin, C., Liang, P., and Hashimoto, T. B. Alpacaeval: An automatic evaluator of instruction-following models, 2023b.	
Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In <i>International Conference on Learning Representations</i> , 2023.	
Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., and Zhu, C. G-eval: Nlg evaluation using gpt-4 with better human alignment. <i>arXiv preprint arXiv:2303.16634</i> , 2023.	
Mialon, G., Fourier, C., Swift, C., Wolf, T., LeCun, Y., and Scialom, T. Gaia: a benchmark for general ai assistants. <i>International Conference on Learning Representations</i> , 2023.	
Miao, N., Teh, Y. W., and Rainforth, T. Selfcheck: Using llms to zero-shot check their own step-by-step reasoning. <i>International Conference on Learning Representations</i> , 2023.	

标题因尺寸过大而被抑制

影响声明

我们的方法具有社会影响，既有积极的一面，也有消极的一面。在积极方面，我们的工作有助于基于大语言模型的多智能体系统的高效开发，使其能够应用于广泛的领域。通过引入故障归因机制并实施相应的改进，这些进展有望显著提升大语言模型多智能体系统的性能。然而，该研究也引入了潜在风险。例如，赋予这些系统修改外部环境的能力（如在计算机上执行代码）可能会导致意想不到的后果。

参考文献

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S.等. GPT-4 技术报告. arXiv 预印本 arXiv:2303.08774, 2023. Chan, C.-M., Chen, W., Su, Y., Yu, J., Xue, W., S. 张, Fu, J., 以及 Liu, Z. Chateval: 通过多智能体辩论改进基于大语言模型的评估器. 国际学习表征会议, 2023. Clemen, R. T. 组合预测: 综述与注释书目. 国际预测杂志, 1989. Cui, G., Yuan, L., Wang, Z., Wang, H., Li, W., He, B., Fan, Y., Yu, T., Xu, Q., Chen, W.等. 通过隐式奖励进行过程强化. arXiv 预印本 arXiv:2502.01456, 2025. Deepseek-ai. Deepseek-r1: 通过强化学习激励大语言模型的推理能力 Deepseek-ai. arXiv 预印本 arXiv:2501.12948, 2025. Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., 以及 Su, Y. Mind2web: 迈向网络通用智能体. 神经信息处理系统进展, 2024年. Fourney, A., G. 班萨尔, Mozannar, H., Tan, C., Salinas, E., Niedtner, F., Proebsting, G., Bassman, G., Gerrits, J., Alber, J.等. Magentic-one: 用于解决复杂任务的通用多智能体系统. arXiv 预印本 arXiv:2411.04468, 2024. Fu, J., Ng, S.-K., Jiang, Z., 以及 Liu, P. Gptscore: 按需评估. 2024年北美计算语言学协会人类语言技术会议论文集, 2023. Ghafarollahi, A. 和 Buehler, M. J. Sciagents: 通过多智能体智能图推理实现科学发现的自动化. arXiv 预印本 arXiv:2409.05556, 2024. Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., et al. A survey on llm-as-a-judge. arXiv 预印本 arXiv:2411.15594, 2024年. Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., et al. Metagpt: Meta programming for multi-agent collaborative framework. 国际学习表征会议, 2023. Hu, Z., Zhang, J., Xiong, Z., Ratner, A., Xiong, H., and Krishna, R. Language model preference evaluation with multiple weak evaluators. arXiv 预印本 arXiv:2410.12869, 2024年. Huang, J. and Chang, K. C.-C. Towards reasoning in large language models: A survey. In Findings of the Association for Computational Linguistics: ACL 2023, 2022年. Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. Large language models cannot self-correct reasoning yet. arXiv 预印本 arXiv:2310.01798, 2023. Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-Bench: Can language models resolve real-world GitHub issues? 国际学习表征会议, 2023. Li, G., Hammoud, H., Itani, H., Khizbullin, D., and Ghanem, B. Camel: Communicative agents for” mind” exploration of large language model society. Advances in Neural Information Processing Systems, 36:51991–52008, 2023a. Li, X., Zhang, T., Dubois, Y., Taori, R., Gulrajani, I., Guestris, C., Liang, P., and Hashimoto, T. B. AlpacaEval: An automatic evaluator of instruction-following models, 2023b. Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In 国际学习表征会议, 2023. Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., and Zhu, C. G-Eval: NLG 评估 using GPT-4 with better human alignment. arXiv 预印本 arXiv:2303.16634, 2023. Mialon, G., Fourier, C., Swift, C., Wolf, T., LeCun, Y., and Scialom, T. GAIA: a benchmark for general AI assistants. 国际学习表征会议, 2023. Miao, N., Teh, Y. W., and Rainforth, T. SelfCheck: Using 大语言模型 to zero-shot check their own step-by-step reasoning. 国际学习表征会议, 2023.

Title Suppressed Due to Excessive Size
Nelson, E., Kollias, G., Das, P., Chaudhury, S., and Dan, S. Needle in the haystack for memory based large language models. <i>arXiv preprint arXiv:2407.01437</i> , 2024.
Scriven, M. <i>Evaluation thesaurus</i> . Sage Publications, 1991.
Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. <i>Advances in Neural Information Processing Systems</i> , 2024.
Song, L., Liu, J., Zhang, J., Zhang, S., Luo, A., Wang, S., Wu, Q., and Wang, C. Adaptive in-conversation team building for language model agents. <i>arXiv preprint arXiv:2405.19425</i> , 2024.
Thakur, A. S., Choudhary, K., Ramayapally, V. S., Vaidyanathan, S., and Hupkes, D. Judging the judges: Evaluating alignment and vulnerabilities in llms-as-judges. <i>arXiv preprint arXiv:2406.12624</i> , 2024.
van Schaik, T. A. and Pugh, B. A field guide to automatic evaluation of llm-generated summaries. In <i>Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval</i> , pp. 2832–2836, 2024.
Wang, F., Zhang, Z., Zhang, X., Wu, Z., Mo, T., Lu, Q., Wang, W., Li, R., Xu, J., Tang, X., et al. A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness. <i>arXiv preprint arXiv:2411.03350</i> , 2024.
Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., Li, Y., Chen, D., Wu, Y., and Sui, Z. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. <i>arXiv preprint arXiv:2312.08935</i> , 2023.
Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. <i>Advances in Neural Information Processing Systems</i> , pp. 24824–24837, 2022.
Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., and Wang, C. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. <i>Conference on Language Modeling</i> , 2023.
Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., et al. Os-world: Benchmarking multimodal agents for open-ended tasks in real computer environments. <i>arXiv preprint arXiv:2404.07972</i> , 2024.
Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. <i>ICLR</i> , 2022.
Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. <i>Advances in Neural Information Processing Systems</i> , 2024.
Yoran, O., Amouyal, S. J., Malaviya, C., Bogin, B., Press, O., and Berant, J. Assistantbench: Can web agents solve realistic and time-consuming tasks? In <i>Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing</i> , 2024.
Yu, T. and Zhu, H. Hyper-parameter optimization: A review of algorithms and applications. <i>arXiv preprint arXiv:2003.05689</i> , 2020.
Zhang, J., Krishna, R., Awadallah, A. H., and Wang, C. Ecoassistant: Using llm assistant more affordably and accurately. <i>arXiv preprint arXiv:2310.03046</i> , 2023a.
Zhang, S., Jia, F., Wang, C., and Wu, Q. Targeted hyper-parameter optimization with lexicographic preferences over multiple objectives. In <i>International Conference on Learning Representations</i> , 2023b.
Zhang, S., Zhang, J., Ding, D., Garcia, M. H., Mallick, A., Madrigal, D., Xia, M., Rühle, V., Wu, Q., and Wang, C. Ecoact: Economic agent determines when to register what action. <i>arXiv preprint arXiv:2411.01643</i> , 2024a.
Zhang, S., Zhang, J., Liu, J., Song, L., Wang, C., Krishna, R., and Wu, Q. Offline training of language model agents with functions as learnable weights. In <i>International Conference on Machine Learning</i> , 2024b.
Zhang, S., Dong, Y., Zhang, J., Kautz, J., Catanzaro, B., Tao, A., Wu, Q., Yu, Z., and Liu, G. Nemotron-research-tool-n1: Tool-using language models with reinforced reasoning. <i>arXiv preprint arXiv:2505.00024</i> , 2025.
Zheng, C., Zhang, Z., Zhang, B., Lin, R., Lu, K., Yu, B., Liu, D., Zhou, J., and Lin, J. Processbench: Identifying process errors in mathematical reasoning. <i>arXiv preprint arXiv:2412.06559</i> , 2024.
Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging llm-as-a-judge with mt-bench and chatbot arena. <i>Advances in Neural Information Processing Systems</i> , 2023.
Zhong, J., Shen, W., Li, Y., Gao, S., Lu, H., Chen, Y., Zhang, Y., Zhou, W., Gu, J., and Zou, L. A comprehensive survey of reward models: Taxonomy, applications, challenges, and future. <i>arXiv preprint arXiv:2504.12328</i> , 2025.
Zhuge, M., Zhao, C., Ashley, D., Wang, W., Khizbullin, D., Xiong, Y., Liu, Z., Chang, E., Krishnamoorthi, R., Tian, Y., et al. Agent-as-a-judge: Evaluate agents with agents. <i>arXiv preprint arXiv:2410.10934</i> , 2024.

标题因尺寸过大而被抑制
E. 尼尔森, G. 科利亚斯, P. 达斯, S. 乔杜里, 和 S. 丹. 基于记忆的大型语言模型的针在草堆中. arXiv 预印本 arXiv:2407.01437, 2024年.
M. 斯科里文. 评估同义词词典. Sage 出版社, 1991.
N. 欣, F. 卡萨诺, A. 戈皮纳斯, K. 纳拉辛汉, 和 S. 姚. Reflexion: 具有语言智能体的语言智能体与言语强化学习. 神经信息处理系统进展, 2024年.
L. 宋, J. 刘, J. 张, S. 张, A. 罗, S. 王, Q. 吴, 和 C. 王. 语言模型智能体的对话中自适应团队构建. arXiv 预印本 arXiv:2405.19425, 2024年.
A. S. 塔库尔, K. 乔杜里, V. S. 拉马亚帕利, S. 韦迪亚纳坦, 和 D. 胡克斯. 评判评判者: 评估大语言模型作为评判者的对齐性和漏洞. arXiv 预印本 arXiv:2406.12624, 2024年.
T. A. 范·斯海克和 B. 帕格. LLM 生成摘要自动评估实用指南. 收录于《第47届 ACM SIGIR 信息检索研究与发展国际会议论文集》, 第2832–2836页, 2024年.
F. 王, Z. 张, X. 张, Z. 吴, T. 莫, Q. 卢, W. 王, R. 李, J. 徐, X. 唐等. 大语言模型时代的小语言模型综合调查: 技术、增强、应用、与大语言模型的协作以及可信度. arXiv 预印本 arXiv:2411.03350, 2024年.
P. 王, L. 李, Z. 邵, R. 徐, D. 戴, Y. 李, D. 陈, Y. 吴和 Z. 随. Math-shepherd: 无需人工标注即可逐步验证和强化 LLM. arXiv 预印本 arXiv:2312.08935, 2023年.
J. 韦, X. 王, D. 舒尔曼斯, M. 博斯玛, F. 夏, E. 齐, Q. V. 李, D. 周, 等. 思维链提示在大型语言模型中激发推理. Advances in NeuralInformation ProcessingSystems, pp. 24824–24837, 2022年.
Q. 吴, G. 班萨尔, J. 张, Y. 吴, S. 张, E. 朱, B. 李, L. 姜, X. 张, 和 C. 王. AutoGen: 通过多智能体对话框架赋能下一代 LLM 应用. 语言建模会议, 2023.
Xie, T., D. 张, J. 陈, X. 李, S. 赵, R. 曹, T. J. Hua, Z. Cheng, D. Shin, F. 雷, 等. Os-world: 在真实计算机环境中对开放式任务的多模态智能体进行基准测试. arXiv 预印本 arXiv:2404.07972, 2024年.
S. 姚, J. 赵, D. 余, N. Du, I. Shafran, K. 纳拉辛汉, 和 Y. Cao. React: 在语言模型中协同推理与行动. ICLR, 2022年.
S. 姚, D. 余, J. 赵, I. Shafran, T. Griffiths, Y. Cao, 和 K. 纳拉辛汉. 思维树: 使用大语言模型进行深思熟虑的问题求解. 神经信息处理系统进展, 2024年. O. Yoran, S. J. Amouyal, C. Malaviya, B. Bogin, O. Press, 和 J. Berant. AssistantBench: 网页智能体能否解决现实且耗时较长的任务? 载于《2024年自然语言处理实证方法会议论文集》, 2024年. T. Yu 和 H. Zhu. 超参数优化: 算法与应用综述. arXiv 预印本 arXiv:2003.05689, 2020年. J. 张, R. Krishna, A. H. Awadallah, 和 C. 王. EcoAssistant: 更经济、更准确地使用大语言模型助理. arXiv 预印本 arXiv:2310.03046, 2023a.
S. 张, F. 家, C. 王, Q. 吴. 针对多目标的字典序偏好的目标超参数优化. 国际学习表征会议, 2023b.
S. 张, J. 张, D. 丁, M. H. Garcia, A. Mallick, D. Madrigal, M. Xia, R`uhle, V., Q. 吴, 和 C. 王. Ecoact: 经济智能体决定何时注册什么动作. arXiv 预印本 arXiv:2411.01643, 2024a.
S. 张, J. 张, J. 刘, L. 宋, C. 王, R. Krishna, 和 Q. 吴. 以作为可学习权重的函数离线训练语言模型智能体. 国际机器学习会议, 2024b.
S. 张, Y. 董, J. 张, J. Kautz, B. Catanzaro, A. Tao, Q. 吴, Z. 余, 和 G. 刘. Nemotron-research-tool-n1: 具有强化推理的工具使用语言模型. arXiv 预印本 arXiv:2505.00024, 2025.
Zheng, C., Z. 张, Zhang, B., Lin, R., Lu, K., Yu, B., Liu, D., Zhou, J., and Lin, J. ProcessBench: 识别数学推理中的过程错误. arXiv 预印本 arXiv:2412.06559, 2024年.
Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Z. 吴, Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., 等. 利用 MT-Bench 和聊天机器人竞技场评判大语言模型作为评判者. Advances in Neural Information Processing Systems, 2023年.
Zhong, J., Shen, W., Y. 李, Gao, S., Lu, H., Chen, Y., Zhang, Y., Zhou, W., Gu, J., and Zou, L. 奖励模型综合综述: 分类、应用、挑战与未来. arXiv 预印本 arXiv:2504.12328, 2025年.
Zhuge, M., Zhao, C., Ashley, D., W. 王, Khizbullin, D., Xiong, Y., Liu, Z., Chang, E., Krishnamoorthi, R., Tian, Y., et al. Agent-as-a-judge: 使用智能体评估智能体. arXiv 预印本 arXiv:2410.10934, 2024年.

Appendix

A. Algorithm Details

A.1. Notations

We then provide more details on the Step-by-Step and Binary Search failure attribution methods. To begin, we define some notations used in the algorithms. We employ Q to denote the query provided to the system. $L = \{l_1, l_2, \dots, l_n\}$ denotes the failure log consisting of n entries where each entry l_i specifies the action taken at time step i by one agent. A^* , s^* denotes the agent responsible for the task failure and the decisive error step respectively.

A.2. Details of Step-by-Step

Algorithm 1 Step-by-Step
Require: Query Q , failure log $L = \{l_1, l_2, \dots, l_n\}$ Ensure: Responsible agent A^* , error step s^* 1: for $i \in \{1, 2, \dots, n\}$ do 2: Provide Q and $\{l_1, \dots, l_i\}$ to LLM 3: if LLM indicates error at step i then 4: $s^* \leftarrow i$ 5: Identify responsible agent A^* in l_i 6: Return A^*, s^* 7: end if 8: end for 9: No error found

A.3. Details of Binary Search

Algorithm 2 Binary Search
Require: Query Q , failure log $L = \{l_1, l_2, \dots, l_n\}$ Ensure: Responsible agent A^* , error step s^* Initialize $low \leftarrow 1, high \leftarrow n$ while $low < high$ do $mid \leftarrow \left\lfloor \frac{low + high}{2} \right\rfloor$ Extract log segment $L' \leftarrow \{l_{low}, l_{low+1}, \dots, l_{mid}\}$ Provide Q and L' to LLM if LLM indicates error in L' then $high \leftarrow mid$ else $low \leftarrow mid + 1$ end if end while $s^* \leftarrow low$, identify responsible agent A^* in l_{s^*} Return A^*, s^*

B. Additional Experiments

B.1. Ablation of Reasoning Prompts

LLMs have shown incredible reasoning ability (Huang & Chang, 2022; Wei et al., 2022; Yao et al., 2024), considering these, in both the all-at-once and step-by-step approaches, we explicitly require the LLMs to not only conduct failure attributions but also specify the reasons for these attributions within the prompt. We don’t include binary search here because it doesn’t

附录

A. 算法细节

A.1. 符号说明

接下来，我们将详细介绍逐步处理和二分搜索故障归因方法。首先，我们定义算法中使用的一些符号说明。我们使用 Q 表示提供给系统的查询。 $L = \{l_1, l_2, \dots, l_n\}$ 表示由 n 个条目组成的失败日志，其中每个条目 l_i 指定了某个智能体在时间步 i 所采取的动作。 A^* 、 s^* 分别表示导致任务失败的智能体以及决定性错误步骤。

A.2. 逐步法细节

算法1 逐步法
输入：查询 Q ，失败日志 $L = \{l_1, l_2, \dots, l_n\}$ 输出：负责的智能体 A^* ，错误步骤 s^* 1: 对于 $i \in \{1, 2, \dots, n\}$ 执行 将 Q 和 $\{l_1, \dots, l_i\}$ 提供给大语言模型 如果大语言模型指示在步骤 i 处发生错误，则 $s^* \leftarrow i$ 在 l_i 中确定负责的智能体 A^* 返回 A^*, s^* 结束如果 8: 结束对于 9: 未找到错误

A.3. 二分搜索的详细信息

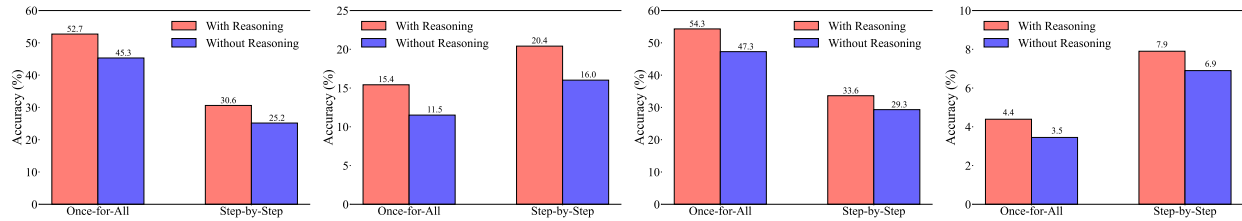
算法 2 二分搜索
需要：查询 Q ，失败日志 $L = \{l_1, l_2, \dots, l_n\}$ 确保：负责智能体 A^* ，错误步骤 s^* 初始化 $low \leftarrow 1, high \leftarrow n$ 当 $low < high$ 时执行 $mid \leftarrow \left\lfloor \frac{low + high}{2} \right\rfloor$ 提取日志片段 $L' \leftarrow \{l_{low}, l_{low+1}, \dots, l_{mid}\}$ 将 Q 和 L' 提供给大语言模型 如果大语言模型在 L' 中指出错误，则 $high \leftarrow mid$ else $low \leftarrow mid + 1$ 结束如果 结束当 $s^* \leftarrow low$ ，在 l_{s^*} 中识别负责的智能体 A^* 返回 A^*, s^*

B. 附加实验

B.1. 推理提示的消融实验

大语言模型展示了令人惊叹的推理能力（黄与张，2022年；Wei等，2022年；Yao等，2024年）。鉴于此，在一次性处理和逐步方法中，我们明确要求大语言模型不仅在提示词中执行故障归因，还要指明这些归因的原因。这里不包含二分搜索，因为它

Title Suppressed Due to Excessive Size



(a) Alg.-Generated Agent-Level (b) Alg.-Generated Step-Level (c) Hand-Crafted Agent-Level (d) Hand-Crafted Step-Level

Figure 7. Ablation of the explicit reasoning prompts in all-at-once and step-by-step. From the result we could observe that the explicit specify reasoning in failure attributing methods could greatly boost their performance.

include reasoning mechanisms in their prompt. We only want binary search to do simple classification task. To investigate the impact of these reasoning prompts on the failure attributions, we conduct additional experiments where the reasoning prompt is removed, allowing the LLMs to directly provide the judgment results. We make comparisons and the results are shown in Figure 7. We observed a significant drop in performance after removing the explicit reasoning prompts for failure attribution in both metrics. For example, in algorithm-generated multi-agent systems, the agent-level accuracy decreased by 7.4% for the all-at-once method. For the step-by-step method, the step-level performance drops 4.4%. These results highlight the necessity of incorporating additional reasoning mechanisms in failure attributions.

C. More Details of Who&When

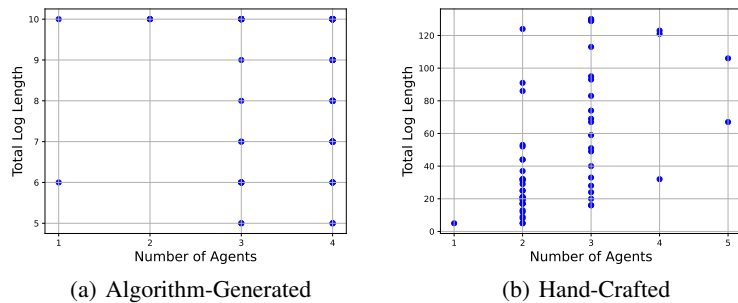
C.1. Overview

	Algorithm-Generated		Hand-Crafted	
	GAIA	AssistantBench	GAIA	AssistantBench
Total Number	98	28	30	28
Maximum Agent Number	4	4	5	4
Minimum Agent Number	1	3	1	2
Maximum Log Length	10	10	130	129
Minimum Log Length	5	6	5	8

Table 5. Additional details about the Who&When benchmark: We present the total number of tasks for each category, along with the maximum and minimum number of agents and log lengths.

We then provide more details about the Who&When dataset, which comprises 184 failure annotations tasks from both hand-crafted and algorithm-generated agentic systems. These failure logs encompass diverse scenarios with varying numbers of agents and interaction lengths. In Table 5, we show the total number of data instances for each category, along with the maximum and minimum number of agents and log lengths. We also visualize the information of each data instance in Figure 8. Note that due to task overlap, some data points may appear sparse in the visualization. We also show an failure task example in Figure 9.

C.2. Data Distribution

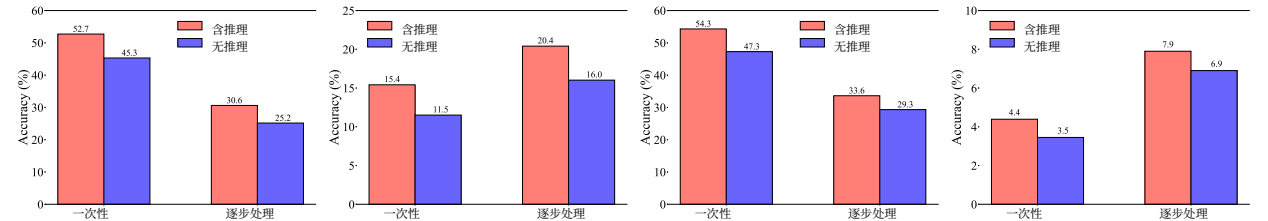


(a) Algorithm-Generated

(b) Hand-Crafted

Figure 8. The number of agents involved and the total length of each failure log instance in the Who&When dataset. Note that due to task overlap, some data points may appear sparse in the visualization

标题因尺寸过大而被抑制



(a) 算法生成的智能体级 (b) 算法生成的步骤级 (c) 人工打造的智能体级

(d) 手工构建的步骤层级

图 7。一次性处理和逐步处理中显式推理提示的消融实验。从结果可以看出，故障归因方法中显式指定推理可以大幅提升其性能。

在其提示词中不包含推理机制。我们只希望二分搜索执行简单分类任务。为了研究这些推理提示词对故障归因的影响，我们进行了附加实验，移除了推理提示词，让大语言模型直接提供判断结果。我们进行了比较，结果如图 7 所示。我们观察到，在两种指标中，移除用于故障归因的显式推理提示词后，性能显著下降。例如，在算法生成的多智能体系统中，一次性方法的智能体级别准确率下降了7.4%。对于逐步方法，步骤层级性能下降了4.4%。这些结果凸显了在故障归因中纳入额外推理机制的必要性。

C. 关于“谁”和“何时”的更多细节

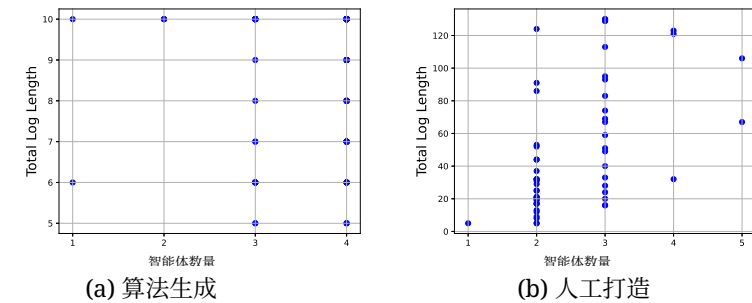
C.1. 概述

	算法生成		人工打造	
	GAIA	AssistantBench	GAIA	AssistantBench
总数	98	28	30	28
最大智能体数量	4	4	5	4
最小智能体数量	1	3	1	2
最大日志长度	10	10	130	129
最小日志长度	5	6	5	8

表5. Who&When基准的附加细节：我们给出了每个类别的任务总数，以及智能体数量和日志长度的最大值与最小值。

随后，我们提供有关 Who&When 数据集的更多细节，该数据集包含来自人工打造和算法生成的智能体系统的 184 个失败标注任务。这些故障日志涵盖了具有不同智能体数量和交互长度的多种场景。在表5中，我们展示了每个类别的数据实例总数，以及智能体数量和日志长度的最大值与最小值。我们还在图8中对每个数据实例的信息进行了可视化。请注意，由于任务重叠，某些数据点在可视化中可能显得较为稀疏。我们还在图9中展示了一个失败任务示例。

C.2. 数据分布



(a) 算法生成

(b) 人工打造

图8。Who&When 数据集中各故障日志实例涉及智能体数量及总长度。注意，由于任务重叠，可视化中部分数据点可能显得稀疏。

C.3. Data Example

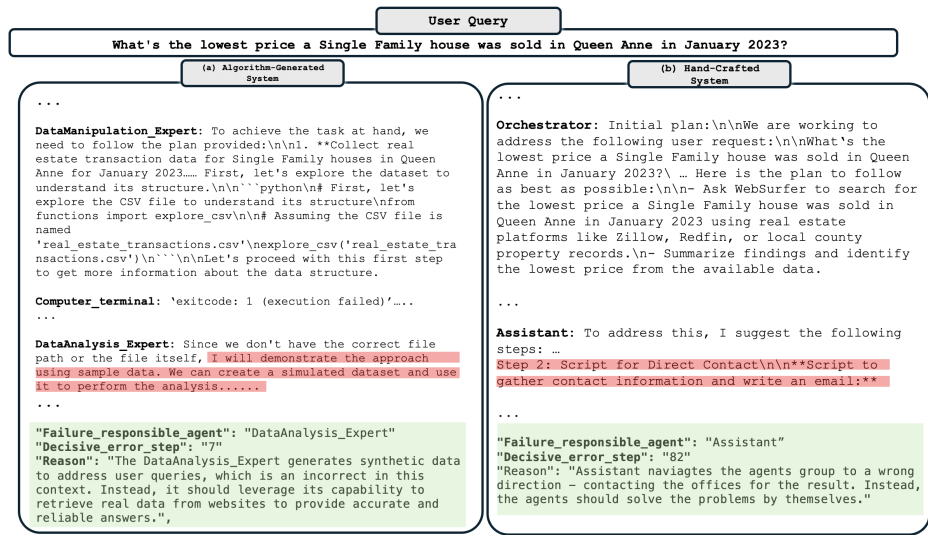


Figure 9. A task example from Who&When, where we annotate failure-responsible agents and their corresponding error steps within the failure logs. Each annotation includes a natural language explanation of the failure reason for reference.

D. Brief Cost Analysis

We then present a brief analysis of the computational costs associated with three failure attribution methods. We focus solely on input tokens, as the contribution of output tokens such as the agent name and error step number is small. We also ignore the mirror token difference between one-time instruction from different methods. We let C to denote the cost of query Q and corresponding instructions of methods. We employ $L = \{l_1, l_2, \dots, l_n\}$, where each entry l_i has an average token of T_l .

D.1. All-at-Once

In the all-at-once method, the LLM receives the full context in a single input. The total input token cost is:

$$Cost_{all-at-once} = C + n \cdot T_l \quad (6)$$

This method incurs the lowest cost as it requires only a single inference step.

D.2. Step-by-Step

In the Step-by-Step method, the LLM processes the failure log incrementally. At each step i , it receives query, instructions and the log segment $\{l_1, \dots, l_i\}$. The process terminates when the decisive error step i^* is found.

$$Cost_{step-by-step} = \sum_{i=1}^{i^*} (C + i \cdot T_l) = i^* \cdot C + T_l \cdot \frac{i^* \cdot (i^* + 1)}{2} \quad (7)$$

In the worst case, $i^* = n$, either when no error is detected or the decisive error occurs in the final step.

D.3. Binary Search

In the Binary Search method, the LLM operates in a logarithmic fashion by iteratively splitting the failure log into halves. At each step i , the segment of the failure log processed by the LLM has a size of approximately $\frac{n}{2^{i-1}}$, where n is the total number of log entries. Therefore the total cost at interaction i is $C + \frac{n \cdot T_l}{2^{i-1}}$. The Binary Search continues until the search space is narrowed down to a single step, requiring $\lceil \log_2(n) \rceil$ iterations. Therefore the cost of binary search is:

C.3. 数据示例

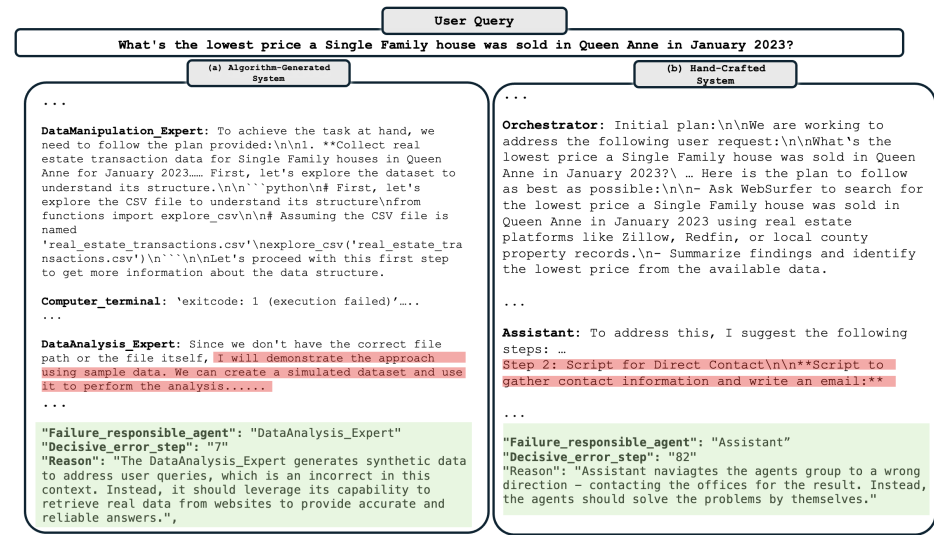


图9. 一个任务示例 from Who&When, 其中我们标注了故障责任智能体及其对应的错误步骤故障日志。每个注释包含故障原因的自然语言解释, 以供参考。

D. 简要成本分析

随后, 我们对三种故障归因方法的计算成本进行了简要分析。我们仅关注输入标记, 因为输出标记 (如智能体名称和错误步骤编号) 所占比重很小。我们也忽略了不同方法中一次性指令之间的镜像标记差异。我们用 C 来表示查询 Q 及相应方法的指令的成本。我们采用 $L = \{l_1, l_2, \dots, l_n\}$, 其中每个条目 l_i 平均包含 T_l 个令牌。

D.1. 一次性处理

在一次性处理的方法中, 大语言模型在单次输入中接收完整上下文。总输入标记成本为:

$$Cost_{all-at-once} = C + n \cdot T_l \quad (6)$$

该方法成本最低, 因为它只需要单个推理步骤。

D.2. 逐步处理

在逐步方法中, 大语言模型增量地处理失败日志。在每个步骤 i , 它接收查询、指令和日志片段 $\{l_1, \dots, l_i\}$ 。当发现决定性错误步骤 i^* 时, 该过程终止。

$$Cost_{step-by-step} = \sum_{i=1}^{i^*} (C + i \cdot T_l) = i^* \cdot C + T_l \cdot \frac{i^* \cdot (i^* + 1)}{2} \quad (7)$$

在最坏情况下, $i^* = n$, 要么未检测到任何错误, 要么决定性错误发生在最后一步。

D.3. 二分搜索

在二分搜索方法中, 大语言模型以对数方式运行, 通过迭代地将失败日志分成两半。在每一步 i , 由大语言模型处理的失败日志片段的大小约为 $\frac{n}{2^{i-1}}$, 其中 n 是日志条目的总数。因此, 在交互 i 时的总成本为 $C + \frac{n \cdot T_l}{2^{i-1}}$ 。二分搜索将持续进行, 直到搜索空间缩小到单一步骤, 这需要 $\lceil \log_2(n) \rceil$ 次迭代。因此, 二分搜索的成本为:

$$Cost_{BinarySearch} = \sum_{i=1}^{\lceil \log_2(n) \rceil} (C + \frac{n \cdot T_l}{2^{i-1}}) = \lceil \log_2(n) \rceil \cdot C + \sum_{i=1}^{\lceil \log_2(n) \rceil} (\frac{n \cdot T_l}{2^{i-1}}) \quad (8)$$

D.4. Cost Summary

In summary, the costs associated with the three methods are influenced by three key factors: the size of the failure log (n), the average token count per log entry (T_l), and the decisive error step (i^*). The choice of method should align with the user’s budget and specific use case requirements. Among the methods, the all-at-once approach incurs the lowest cost as it requires only a single inference step. In contrast, the costs of the binary search and step-by-step methods are highly dependent on the specific scenario, particularly the distribution of decisive error locations and the total length of the failure log.

E. Hyperparameters

Hyperparameters play a critical role in determining the performance of machine learning algorithms (Yu & Zhu, 2020; Zhang et al., 2023b). In this paper, the hyperparameters we utilize are divided into two categories: those used for Who&When data construction and those employed for automated failure attribution algorithms. For data construction, we adopt the default settings of CaptainAgent and Magentic-One from their official libraries (AG2 and Autogen). One notable setting is that the maximum iteration count for CaptainAgent is limited to 10, whereas Magentic-One allows up to 30 rounds. It is important to highlight that the agent’s thought processes are excluded from the round count, which contributes to longer failure log lengths, as discussed in Appendix C. For the inference hyperparameters of other large language models (LLMs), we adhere to the default configurations specified in their official documentation.

F. Annotation Details

In Figure 10, we present our standardized annotation guidelines used by all annotators. The guidelines clearly define criteria for identifying failure-responsible agents and decisive error steps. Annotators are instructed to document any uncertainties in their annotations for subsequent group discussion and voting.

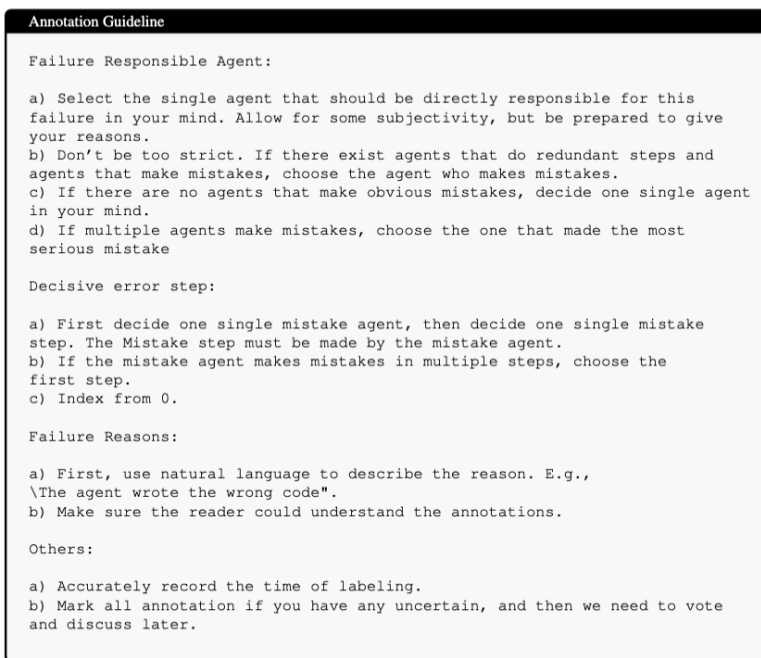


Figure 10. The guideline in making annotation. We maintain consistent annotation guidelines across all annotators.

$$Cost_{BinarySearch} = \sum_{i=1}^{\lceil \log_2(n) \rceil} (C + \frac{n \cdot T_l}{2^{i-1}}) = \lceil \log_2(n) \rceil \cdot C + \sum_{i=1}^{\lceil \log_2(n) \rceil} (\frac{n \cdot T_l}{2^{i-1}}) \quad (8)$$

D.4. 成本摘要

总而言之，三种方法的成本受三个关键因素影响：失败日志的大小（ n ）、每个日志条目的平均令牌数量（ T_l ），以及决定性错误步骤（ i^* ）。方法的选择应与用户的预算和特定用例需求相符。在这些方法中，一次性方法的成本最低，因为它只需要一个推理步骤。相比之下，二分搜索方法和逐步方法的成本在很大程度上取决于具体场景，特别是决定性错误位置的分布以及失败日志的总长度。

E. 超参数

超参数在决定机器学习算法的性能方面起着至关重要的作用（Yu & Zhu, 2020; Zhang et al., 2023b）。在本文中，我们使用的超参数分为两类：一类用于Who&When 数据构建，另一类用于自动故障归因算法。在数据构建方面，我们采用 CaptainAgent 和 Magentic-one 官方库（AG2 和 Autogen）的默认设置。一个值得注意的设置是，CaptainAgent 的最大迭代次数限制为 10，而 Magentic-one 允许最多 30 轮。需要强调的是，智能体的思维过程不计入轮次，这导致了更长的故障日志长度，正如附录 C 中讨论的那样。对于其他大语言模型 (LLMs) 的推理超参数，我们遵循其官方文档中指定的默认配置。

F. 标注细节

图10 展示了所有标注员使用的标准化标注指南。该指南明确定义了识别故障责任智能体和决定性错误步骤的标准。指示标注员在标注中记录任何不确定性，以便随后进行小组讨论和投票。

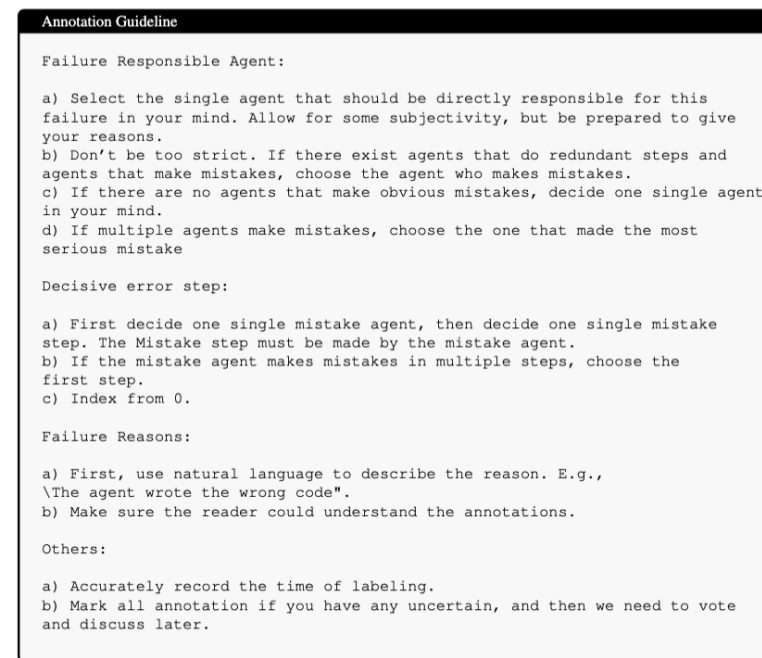


图 10 . 进行标注的指南。我们在所有标注者之间保持一致的标注指南

G. Prompts

We list the prompt templates for all three attribution methods in this section. Please refer to our code base for more details.

G.1. Prompts of All-at-Once

You are an AI assistant tasked with analyzing a multi-agent conversation history when solving a real world problem. The problem is: {problem}. Identify which agent made an error, at which step, and explain the reason for the error. Here’s the conversation: {failure log} Based on this conversation, please predict the following:

1. The name of the agent who made a mistake that should be directly responsible for the wrong solution to the real world problem. If there are no agents that make obvious mistakes, decide one single agent in your mind. Directly output the name of the Expert.
2. In which step the mistake agent first made mistake. For example, in a conversation structured as follows: { “agent a”: ”xx”, “agent b”: ”xxxx”, “agent c”: ”xxxxx”, “agent a”: ”xxxxxxx” }, each entry represents a ’step’ where an agent provides input. The ’x’ symbolizes the speech of each agent. If the mistake is in agent c’s speech, the step number is 2. If the second speech by ’agent a’ contains the mistake, the step number is 3, and so on. Please determine the step number where the first mistake occurred.
3. The reason for your prediction. Please answer in the format:
Agent Name: (Your prediction)
Step Number: (Your prediction)
Reason for Mistake: (Your reason)

You are an AI assistant tasked with analyzing a multi-agent conversation history when solving a real world problem. The problem is: {problem}. The Answer for the problem is: {ground truth}. Identify which agent made an error, at which step, and explain the reason for the error. Here’s the conversation: {failure log} Based on this conversation, please predict the following:

1. The name of the agent who made a mistake that should be directly responsible for the wrong solution to the real world problem. If there are no agents that make obvious mistakes, decide one single agent in your mind. Directly output the name of the Expert.
2. In which step the mistake agent first made mistake. For example, in a conversation structured as follows: { “agent a”: ”xx”, “agent b”: ”xxxx”, “agent c”: ”xxxxx”, “agent a”: ”xxxxxxx” }, each entry represents a ’step’ where an agent provides input. The ’x’ symbolizes the speech of each agent. If the mistake is in agent c’s speech, the step number is 2. If the second speech by ’agent a’ contains the mistake, the step number is 3, and so on. Please determine the step number where the first mistake occurred.
3. The reason for your prediction. Please answer in the format:
Agent Name: (Your prediction)
Step Number: (Your prediction)
Reason for Mistake: (Your reason)

G. 提示词

我们列出提示词模板 本节中三种归因方法的细节。更多详情请参阅我们的代码库。 s.

G.1. 一次性处理的提示词

你是一个AI助手，负责在解决现实世界问题时分析多智能体会话历史。该问题是： {problem}。确定哪个智能体犯了错误，在哪一步骤，并解释该错误的原因。以下是对话： {failure log}基于此对话，请预测以下内容： 1. 犯下导致该现实世界问题出现错误解决方案的直接责任错误的智能体的名称。如果没有智能体犯下明显错误，请你在心中确定一个智能体。直接输出专家的代理名称。2. 犯错的智能体首次犯错的步骤。例如，在如下结构的对话中： {”agent a”: ”xx”, ”agent b”: ”xxxx”, ”agent c”: ”xxxxx”, ”agent a”: ”xxxxxxx”}, 每个条目代表智能体提供输入的“步骤”。“x”符号表示每个智能体的发言。如果错误在agent c的发言中，则步骤编号为2。如果“agent a”的第二次发言中包含错误，则步骤编号为3，依此类推。请确定首次发生错误的步骤编号。3. 你预测的原因。请以以下格式作答：智能体名称：（你的预测）步骤编号：（你的预测）错误原因：（你的原因）

你是一个AI助手，负责在解决现实世界问题时分析多智能体会话历史。问题是： {problem}。该问题的答案是： {ground truth}。识别出哪个智能体犯了错误，在哪个步骤，并解释错误的原因。以下是对话内容： {failure log}根据此对话，请预测以下内容： 1. 犯了错误并应直接对解决现实世界问题的错误解决方案负责的智能体名称。如果没有智能体犯有明显的错误，请在你的心中确定一个单一的智能体。直接输出专家的名称。2. 犯错智能体首次犯错的步骤。例如，在如下结构的对话中： {”agent a”: ”xx”, ”agent b”: ”xxxx”, ”agent c”: ”xxxxx”, ”agent a”: ”xxxxxxx”}, 每个条目代表一个‘步骤’，由一个智能体提供输入。’x’符号代表每个智能体的发言。如果错误发生在agent c的发言中，则步骤编号为2。如果’agent a’的第二次发言包含错误，则步骤编号为3，依此类推。请确定首次发生错误的步骤编号。3. 你预测的理由。请按以下格式回答：智能体名称：（你的预测）步骤编号：（你的预测）错误原因：（你的理由）

G.2. Prompts of Binary Search

You are an AI assistant tasked with analyzing a segment of a multi-agent conversation. Multiple agents are collaborating to address a user query, with the goal of resolving the query through their collective dialogue. Your primary task is to identify location of the most critical mistake, and determine the single step in the conversation where this error occurs, ultimately leading to the failure in resolving the user’s query.

The problem to address is as follows: {problem}.

Review the following conversation range

{range description}: {sliced log}.

Based on your analysis, predict whether the error is more likely to be located in the upper or lower half of the segment. lower half is defined as the range lower half range and upper half is defined as the range upper half range. Please simply output either ’upper half’ or ’lower half’.

You should not output anything else.

You are an AI assistant tasked with analyzing a segment of a multi-agent conversation. Multiple agents are collaborating to address a user query, with the goal of resolving the query through their collective dialogue. Your primary task is to identify location of the most critical mistake, and determine the single step in the conversation where this error occurs, ultimately leading to the failure in resolving the user’s query.

The problem to address is as follows: {problem}.

The Answer for the problem is: {ground truth}.

Review the following conversation range

{range description}: {sliced log}.

Based on your analysis, predict whether the error is more likely to be located in the upper or lower half of the segment. lower half is defined as the range lower half range and upper half is defined as the range upper half range. Please simply output either ’upper half’ or ’lower half’.

You should not output anything else.

G.3. Prompts of Step-by-Step

You are an AI assistant tasked with evaluating the correctness of each step in an ongoing multi-agent conversation aimed at solving a real-world problem.

The problem being addressed is: {problem}.

Here is the conversation history up to the current step: {failure log}.

Your task is to determine whether the most recent agent’s action contains an error that could hinder the problem-solving process. Please respond with ’Yes’ or ’No’ and provide a clear explanation for your judgment.

Note: Please avoid being overly critical in your evaluation.

Attention: Respond in the format:

1. Yes/No. 2. Reason for the judgment.

You are an AI assistant tasked with evaluating the correctness of each step in an ongoing multi-agent conversation aimed at solving a real-world problem.

The problem being addressed is: {problem}.

Here is the conversation history up to the current step: {failure log}.

The Answer for the problem is: {ground truth}.

Your task is to determine whether the most recent agent’s action contains an error that could hinder the problem-solving process. Please respond with ’Yes’ or ’No’ and provide a clear explanation for your judgment.

Note: Please avoid being overly critical in your evaluation.

Attention: Respond in the format:

1. Yes/No. 2. Reason for the judgment.

G.2. 二分搜索提示词

你是一个AI助手，负责分析多智能体对话的一个片段。多个智能体正在协作以解决用户查询，目标是通过他们的集体对话来解决该查询。你的主要任务是识别最关键错误的位置，并确定对话中发生此错误的单一步骤，最终导致解决用户查询的失败。要解决的问题如下： {problem}。请审查以下对话范围{range description}: {sliced log}。根据您的分析，预测该错误更可能位于片段的 上半部分 或 下半部分 。下半部分 定义为范围 下半部分 范围，上半部分 定义为范围 上半部分 范围。请仅输出 ’上半部分’ 或 ’下半部分’。您不应输出任何其他内容。

您是一个AI助手，负责分析一段 多智能体对话 的片段。多个 智能体 正在协作以处理一个 用户查询 ，其目标是通过 集体对话 来解决该 查询 。您的主要任务是定位最关键的 错误 所在位置，并确定 对话 中发生此 错误 的单一 步骤 ，最终导致无法解决 用户查询 。需要解决的问题如下： {problem}。该问题的真值为： {ground truth}。审查以下 对话 范围{range description}: {sliced log}。

根据您的分析，预测该错误更可能位于片段的 上半部分 或 下半部分 。下半部分 定义为范围 下半部分 范围，上半部分 定义为范围 上半部分 范围。请仅输出 ’上半部分’ 或 ’下半部分’。您不应输出任何其他内容。

G.3. 逐步处理提示词

你是一名AI助手，负责评估旨在解决现实世界问题的正在进行的智能体对话中每一步的正确性。所要解决的问题是： {问题}。以下是截至当前步骤的对话历史： {失败日志}。你的任务是判断最近一个智能体的动作是否包含可能阻碍问题求解过程的错误。请以“是”或“否”作答，并针对你的判断提供清晰的解释。注意：在评估时请避免过于苛刻。注意：请按以下格式回答： 1. 是/否。 2. 判断理由。

您是一个AI助手，负责评估旨在解决一个现实世界问题的正在进行的多智能体对话中每个步骤的正确性。所解决的问题是： {problem}。以下是直到当前步骤的对话历史： {failure log}。该问题的答案是： {ground truth}。您的任务是判断最近的智能体的动作是否包含可能阻碍问题解决过程的错误。请用“是”或“否”回答，并对您的判断提供清晰的解释。注意：在评估中请避免过于苛刻。注意：请按以下格式回答： 1. 是/否。 2. 判断理由。